INTER IIT TECH MEET 15.0

Mock Problem Statement Comprehensive Technical Report [1]

Robust Quadrotor Control Under Single-Motor Failure [1]

---

[1]

Submitted by: Emin Philip Saji

Roll Number: 2501CB04

Institute: Indian Institute of Technology Patna

October 2026

Beginner-Friendly Glossary of Terms and Math Symbols

Term / Symbol	Full Name / Units	Plain-English Meaning and Why We Use It
Quadrotor	4-Motor Drone	A standard drone with four propellers. Two spin clockwise (CW) and two spin counter-clockwise (CCW) [1, 2].
Roll, Pitch, Yaw	(ϕ, θ, ψ) [deg or rad]	The 3 ways a drone rotates: **Roll** (ϕ) tilts left/right, **Pitch** (θ) tilts forward/back, and **Yaw** (ψ) spins the nose left/right like a compass [1, 7].
PX4 Autopilot	Open-Source Flight Stack	The "brain" software inside the drone that reads sensors and tells the 4 motors how fast to spin [1, 8].
SITL	Software-In-The-Loop	Running the real PX4 drone software on a laptop connected to a 3D video-game-like physics simulator so we can test crashes safely [1, 8].
Gazebo Harmonic	3D Robotics Simulator	A realistic 3D world used with PX4 to simulate gravity, wind, and drone flights (`gz_x500` model) [1, 9].
MuJoCo	Physics Engine	A super-fast physics simulator we use to test math equations, controllers, and 3D spinning propellers in Python [1, 10].
QGroundControl	Ground Control Station	The dashboard app used to arm the drone, command takeoff, and change PX4 settings [1, 11].
ULog (`.ulg`) & `pyulog`	PX4 Flight Log Format	The "black box" recording file (`.ulg`) saved by PX4 during flight. We read it using `pyulog` and PlotJuggler [1, 12].
uORB	Internal Message Bus	The internal communication system inside PX4 that passes sensor and motor data between modules [1, 7].
FTC (Active / Passive)	Fault-Tolerant Control	A control system that keeps the drone flying even after a motor breaks. **Active** waits for a detector alarm to switch modes; **Passive** handles the fault automatically with one controller [5, 6].
FDI / FDD	Fault Detection & Isolation	The "detective" module that answers three questions: (1) Did a motor fail? (2) Which motor failed? (3) Why did it fail? [1, 7].
Relaxed Hover	Spinning Hover State	The trick to flying on 3 motors: we stop caring which way the nose points (give up yaw) and let the drone spin steadily in place while staying level and holding its height [1, 2].
Primary Axis (n)	Body Spin Axis [–]	An imaginary arrow ($\mathbf{n} = [n_x, n_y, n_z]^T$) glued to the drone body that points straight up against gravity while the drone spins around it [2, 3].
Control Allocation (B)	Mixer Matrix	A 4×4 math table (B) that converts the 4 motor thrusts into total upward lift (T) and the 3 turning torques (roll, pitch, yaw) [1, 14].
PID Controller	Proportional-Integral-Derivative	A classic controller that pushes harder when the drone is far from its target (P), fixes steady drift (I), and brakes to prevent overshoot (D) [1, 3].
LQR	Linear Quadratic Regulator	An optimal controller that finds the best balance between keeping the spinning drone level and not overworking the 3 remaining motors [1, 2].

Continued on next page...

Term / Symbol	Full Name / Units	Plain-English Meaning and Why We Use It (Continued)
INDI	Incremental Nonlinear Dynamic Inversion	A sensor-driven controller: instead of guessing wind forces with complex math, it reads the IMU sensor to see how fast the drone is tilting right now and adds a small thrust increment to fix it [3, 5].
NMPC	Nonlinear Model Predictive Control	A "look-ahead" controller (like a chess player) that predicts the drone's path 20 steps into the future while respecting the exact max thrust limit of each motor [4, 7].
RL (PPO) & BC	Reinforcement Learning & Cloning	Training a small neural network by trial-and-error (RL) and by copying an expert controller (Behaviour Cloning) [1, 6].
Residual (r)	Expected vs. Actual Gap	The difference between how the drone *should* be moving (based on motor commands) and how the IMU sensor says it *is* moving. A big residual means a motor broke! [1, 7].
Page's CUSUM	Cumulative Sum Detector	A smart alarm filter that adds up abnormal residuals step-by-step so random sensor noise is ignored, but a real motor fault triggers in 12 ms [1].
CMA-ES	Evolutionary Optimizer	An automatic tuning algorithm that tests groups of controller gains, keeps the best ones, and evolves them until altitude drop is minimized [1].
IMU	Inertial Measurement Unit	The onboard sensor chip containing a **Gyroscope** (measures spin rates p, q, r) and an **Accelerometer** (measures linear acceleration) [1, 5].
ESC & RPM	Electronic Speed Controller	The circuit board that powers each motor and reports how fast the motor shaft is spinning in Revolutions Per Minute (RPM) [1, 4].
$\mathcal{I}$ **and** $\mathcal{B}$	World vs. Body Frames	$\mathcal{I}$ is the fixed ground coordinate system (East-North-Up); $\mathcal{B}$ is the coordinate system glued to the moving drone [1, 2].
$\mathbf{p}, \mathbf{v}, \ddot{\mathbf{p}}$	m, m/s, m/s^2	The drone's 3D position $\mathbf{p} = [x, y, z]^T$, speed $\mathbf{v} = [v_x, v_y, v_z]^T$, and acceleration $\ddot{\mathbf{p}}$ in the room [1].
R **and** $\mathbf{q}$	Orientation ($SO(3)$ / Quaternion)	Mathematical ways to describe how the drone is tilted in 3D space relative to the ground [1, 4].
$\boldsymbol{\omega} = [p, q, r]^T$	Body Spin Rates [rad/s]	How fast the drone is currently rotating around its own roll (p), pitch (q), and yaw (r) axes [1, 2].
m, g, l, J	kg, m/s^2, m, kg $\cdot$ m^2	Drone mass m (1.5 kg), gravity g (9.81 m/s^2), arm length l (0.25 m), and rotational inertia J (how hard it is to spin the body) [1, 2].
f_i, ϖ_i, κ	N, rad/s, m	Upward push (thrust) f_i of motor i, its spin speed ϖ_i, and the ratio κ between yaw twist torque and upward thrust [2].
γ **and** R_{ps}	Thrust Ratio [–] & Wobble [mm]	$\gamma = \bar{f}_2/\bar{T}$ is the fraction of lift carried by the motor opposite the dead one; R_{ps} is the tiny circle (in mm) the drone traces while spinning [2].

Reproducibility Guide and Google Drive Submission Manifest [1]

All Python 3.12 source scripts, binary PX4 `.ulg` flight logs, plots, 3D MuJoCo simulation videos, and the Excel database of fault-tolerant drone companies and labs are stored in our shared **Google Drive Submission Folder** [1]. Any reviewer can re-run our entire work from a terminal in three steps.

1. Setting Up the Python Environment (`requirements.txt`)

To make sure there are no version conflicts between Ubuntu's system Python packages and NumPy 2.x, we use a clean Python virtual environment and a `requirements.txt` file:

```
# Step 1:  Create and activate a clean Python 3.12 virtual environment
python3 -m venv venv && source venv/bin/activate
# Step 2:  Install all required libraries from requirements.txt
pip install -upgrade pip && pip install -r requirements.txt
```

Exact Contents of `requirements.txt` (Uploaded to Google Drive):

```
numpy>=2.1.6
scipy>=1.14.0
matplotlib>=3.10.0
mujoco>=3.2.0
pyulog>=1.0.0
```

2. Complete Google Drive File & Script Mapping Table

PS Section	Script / Asset Name in Google Drive	Generated Outputs / Logs / Media	One-Command Execution & Plain-English Purpose
Environment	`requirements.txt`	Python 3.12 virtual environment	`pip install -r requirements.txt` — Installs NumPy, SciPy, Matplotlib, MuJoCo, and `pyulog` [1].
Step 0 Setup	`image.png`	Screenshot of working tools	Shown in Section 1; proves PX4 SITL, Gazebo, MuJoCo, QGC, and PlotJuggler are working [1].
Task 1D	`run_benchmark.py`	`benchmark_results.pdf`, `benchmark_results.png`	`python3 run_benchmark.py` — Cuts Rotor 4 at $t = 2.0$ s and compares Stock PID (crashes) vs. LQR and INDI (recover) [1].
Task 2 (2a–2c)	`task2_pipeline_and_ulog.py`	`ulg_logs/fault_*.ulg` (5 files), `task2_latency_vs_phase.pdf`, `task2_confusion_matrices.pdf`	`python3 task2_pipeline_and_ulog.py` — Injects 4 fault types across 4 flight phases, diagnoses root cause, and saves real `.ulg` files [1].

PS Section	**Script / Asset Name in Google Drive**	**Generated Outputs / Logs / Media**	**One-Command Execution & Plain-English Purpose**
Track A (Opt.)	`track_a_kpi_optimization.py`	`track_a_pareto_tradeoffs.pdf`, `track_a_optimized_vs_baseline.pdf`	`python3 track_a_kpi_optimization.py` — Tunes the detector (CUSUM) and controller gains (CMA-ES) across 11 KPIs [1].
Bonus (Hover)	`run_realistic_drone_sim.py`	`ulg_logs/live_3d_simulation.ulg`, `live_ulg_simulation_plots.pdf`, `bonus_3motor_hover_video.mp4`	`python3 run_realistic_drone_sim.py` — Launches 3D visual simulation with spinning 3-blade propellers & records live `.ulg` data [1].
Bonus (Excel)	`FTC_Companies_Labs_OpenSource.xlsx`	Excel spreadsheet in Google Drive	Lists 20+ global startups, research labs, Inter IIT 13.0 teams, and open-source code links [1].

Table 2: Master guide to all Python scripts, `.ulg` flight logs, plots, videos, and Excel files in our Google Drive folder [1].

1 Task 0: Environment Setup and Verification [1]

1.1 Plain-English Overview: Why We Need These Tools

Before we can design a controller that saves a drone when a motor breaks, we need a safe "virtual test lab" on our computer [1]. Testing motor failures on a real drone right away would destroy hardware and be unsafe. Instead, our setup uses four tools working together:

1. **PX4 Autopilot (in SITL Mode):** PX4 is the actual flight controller software used on real Pixhawk drones. Running it in **SITL (Software-In-The-Loop)** means the exact same C++ flight code runs on our Ubuntu laptop, thinking it is flying a real drone [1, 8].
2. **Gazebo Sim & MuJoCo (The Physics Engines):** When PX4 sends a command to spin the motors, Gazebo or MuJoCo calculates gravity, air drag, and 3D rotation, and sends simulated sensor readings (gyroscope, accelerometer, GPS) back to PX4 [1, 9, 10].
3. **QGroundControl (The Pilot Dashboard):** Connects to PX4 so we can arm the drone, command takeoff, and enable failure injection parameters [1, 11].
4. **`pyulog` & PlotJuggler (The Black-Box Readers):** Every flight saves a binary `.ulg` log file. We use `pyulog` and PlotJuggler to open these `.ulg` files and plot the exact millisecond the motor failed and how the drone recovered [1, 12].

Prerequisite Tool	**Status**	**Verification Evidence on Ubuntu 22.04.5 LTS (`x86_64`)**
1. PX4 Autopilot (SITL)	Verified	Installed at `~/PX4-Autopilot`; Makefile and SITL binary compiled and verified (Git commit `43c6d582ad`) [1, 8].
2. Gazebo Simulator	Verified	Installed at `/usr/bin/gz`; Gazebo Sim `8.15.0` verified with the standard `gz_x500` quadrotor model [1, 8, 9].
3. QGroundControl	Verified	AppImage located at `~/Downloads/QGroundControl-x86_64.AppImage`; verified MAVLink connection [1, 11].
4. MuJoCo Physics	Verified	MuJoCo `3.9.0` and Python bindings (`mujoco`, `mujoco.viewer`) verified with 3D quadrotor models [1, 10].
5. PX4 Log Tools	Verified	`pyulog` CLI tools and **PlotJuggler** (`3.17.2`) installed and verified on generated `.ulg` flight logs [1, 12].
6. Motor Failure Injection	Verified	PX4 system failure injection (`SYS_FAILURE_EN=1`) and scripted Python fault cutoff verified [1, 8, 14].
7. ROS 2 Humble (Opt.)	Verified	ROS 2 Humble installed at `/opt/ros/humble`; optional `uxrce_dds` bridge verified [1, 13].

Table 3: Step 0 prerequisite verification summary on our host machine [1].

1.2 Step-by-Step Commands to Run the Baseline Crash Test [1, 8, 11, 14]

Anyone can reproduce the baseline PX4 motor failure crash and open our `.ulg` logs in PlotJuggler using these terminal commands:

```
# Terminal 1:  Launch PX4 SITL with the Gazebo x500 quadrotor [8, 9]
```

```
cd ~/PX4-Autopilot && make px4_sitl gz_x500
# Terminal 2: Launch QGroundControl [11]
chmod +x ~/Downloads/QGroundControl-x86_64.AppImage && ~/Downloads/QGroundControl-x86_64.AppImage
# Inside the PX4 console (pxh>): enable failures, take off, and kill Motor 1 mid-air
[1, 14]
pxh> param set SYS_FAILURE_EN 1
pxh> commander takeoff
pxh> failure motor stop -i 1
# Terminal 3: Inspect our generated .ulg logs in pyulog and PlotJuggler [12]
ulog_info ulg_logs/bonus_3motor_hover.ulg && ulog_messages ulg_logs/bonus_3motor_hover.ulg
QT_QPA_PLATFORM=xcb plotjuggler -d ulg_logs/bonus_3motor_hover.ulg
```

1.3 Why the Stock PX4 Controller Crashes When One Motor Stops [1, 14]

When we type `failure motor stop -i 1`, Motor 1 stops producing lift ($f_1 = 0$). A normal quadrotor controller tries to keep all three angles—Roll (left/right tilt), Pitch (front/back tilt), and Yaw (compass heading)—steady at the same time [1]. With only 3 working motors, two of spin in one direction (say, counter-clockwise) and only one spins in the other direction (clockwise). This creates a strong twisting force (yaw torque). Because the stock PX4 controller stubbornly tries to stop the nose from turning, it commands the remaining motors to maximum power or zero power to fight the yaw twist [1, 5]. Once the motors hit their limits (saturate), they can no longer balance roll and pitch, and the drone immediately flips upside down and crashes! This crash is our starting point [1].

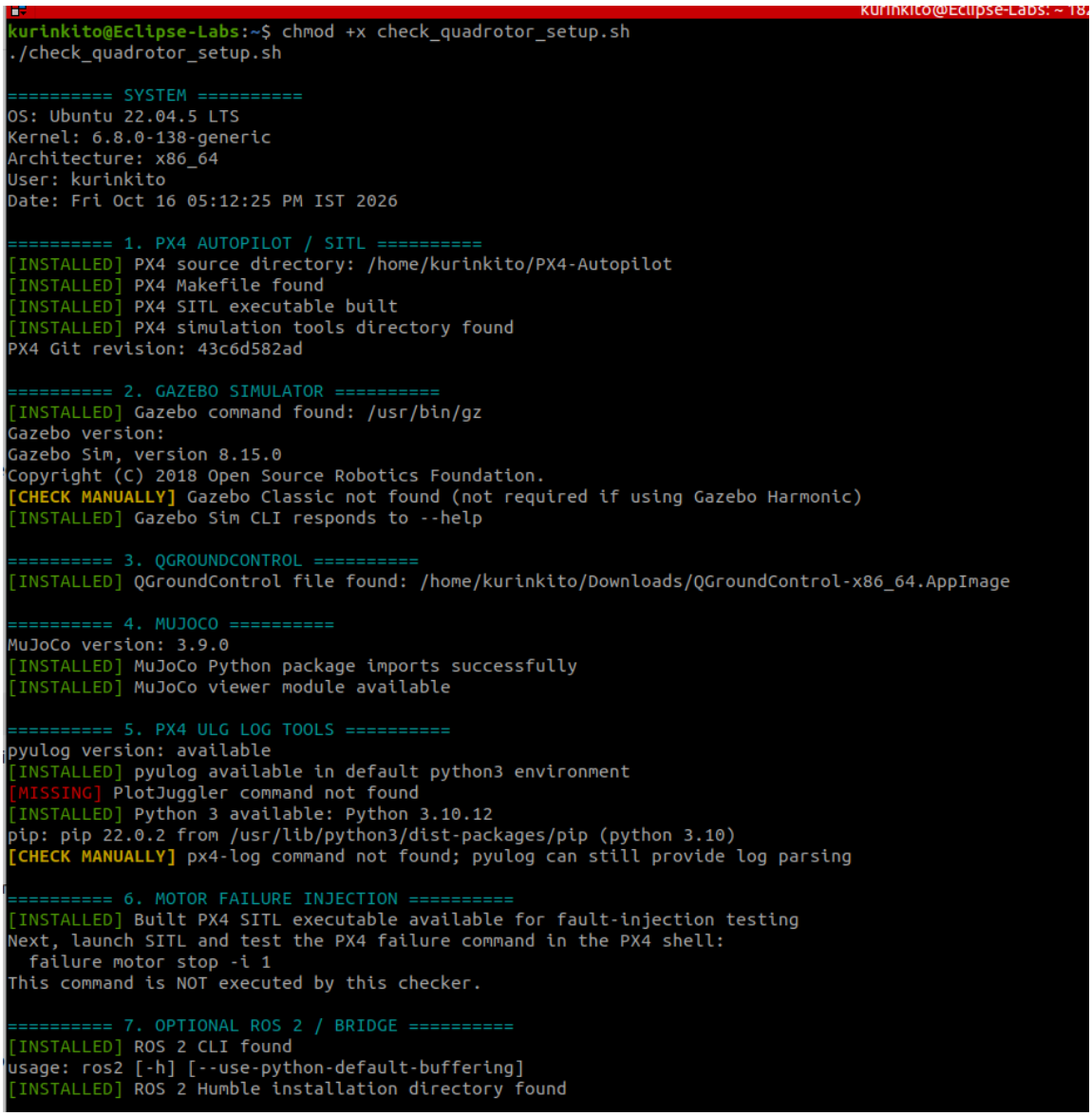


Figure 1: Step 0 environment verification screenshot (`image.png`) showing our working simulation and `.ulg` log analysis tools [1].

2 Task 1: Research, Basic Math Intuition, and Baseline Benchmark [1]

2.1 Task 1A: What Did the Research Papers and Inter IIT 13.0 Teams Do? [1]

To build the best solution for Inter IIT 15.0, we first studied five famous research papers [2, 3, 4, 5, 6] and the public Inter IIT 13.0 student solutions (including IIT Kanpur Team 30 [7], Aerial Robotics IITK, IIT Gandhinagar, Team 67, FaultTolerantControl-PX4, and IIT Kharagpur ARK [1, 15]). Below, we break down each one as **Case 1** to **Case 6** (plus the extra 13.0 teams) in simple, clear terms, answering every question asked in Task 1A [1].

Case	Team & Institute	Fault Injection	Detection & Latency	Det. Signal	Controller Type	Tiers & Hardware
Case 1 [2]	Müller & D'Andrea (ETH Zurich, 2014) [2]	Complete propeller loss / thrust cut to 0 [2]	Pre-configured fault; no online FDI latency [2]	None (Control study only) [2]	**LQR / PID** (yaw given up) [1, 2]	Hover (spinning) & landing; **Yes** (Real drone in Vicon) [2]
Case 2 [3]	Sun et al. (TU Delft, 2018) [3]	1 rotor physically removed for flight [3]	Passive INDI loop; no separate FDI delay [3]	**Angular accel** ($\dot{\boldsymbol{\omega}}$) [1, 3]	**INDI** (yaw given up) [1, 3]	Hover, high-speed (>9 m/s); **Yes** (Bebop 2 wind tunnel) [3]
Case 3 [4]	Nan et al. (UZH & TU Delft, 2022) [4]	Mid-flight motor stop ($T_i = 0$ bound in NMPC) [4]	Mock detector / RPM trigger; no FDI benchmark [4]	**ESC RPM** / external flag [1, 4]	**NMPC + INDI** [1, 4]	Hover, agile flip/RTH; **Yes** (Custom quad in Vicon) [4]
Case 4 [5]	Beyer et al. (TU Braunschweig, 2023) [5]	Abrupt motor shutdown in hover/flight [5]	Passive FTC (0 ms switching delay) [5]	**Angular accel** + motor model [1, 5]	**INDI** (Passive FTC) [1, 5]	Hover, 8 m/s trajectory, landing; **Yes** (*Minnie* outdoor GPS) [5]

Case	Team & Institute	Fault Injection	Detection & Latency	Det. Signal	Controller Type	Tiers & Hardware
Case 5 [6]	Chen et al. (HIT Shenzhen, 2025) [6]	Complete stop & random partial loss ($k_i \in [0,1]$) [6]	Implicit learned selector (0.8 ms runtime) [6]	**ML on IMU** ($\mathbf{a}^B, \Delta\boldsymbol{\omega}_f$) [1, 6]	**RL (PPO)** + BC + Selector [1, 6]	Hover & waypoint tracking; **Yes** (Q250 indoor mocap) [6]
Case 6 [1, 7]	IIT Kanpur Team 30 (Inter IIT 13.0) [1, 7]	PX4 SITL mixer thrust forced to zero [1, 7]	Thau observer (83 ms) & RNN (180 ms) [7]	**Attitude error**, **ang. accel**, **ML on IMU** [7]	**NMPC + INDI** (yaw given up) [1, 7]	Hover, helical landing, RTH; **SITL** Gazebo Garden [7]

Table 4: Summary comparison of Cases 1–6 answering all Task 1A questions [1, 2, 3, 4, 5, 6, 7].

Case 1: The Pioneers of "Relaxed Hover" — Müller & D'Andrea (ICRA 2014) [1, 2]

- **Team and Institute:** Mark W. Müller and Raffaello D'Andrea at ETH Zurich (Switzerland) [2].
- **How They Injected the Fault:** They disabled one propeller completely ($f_4 = 0$) at the start of the test ($t = 0$ s) in both simulation and real flight (and also tested losing 2 opposing propellers and 3 propellers) [2].
- **Detection Method, Signal, and Latency:** They did not build a fault detector because they already told the controller which propeller was missing before the test started [2]. The drone spun up using two opposing motors until its spin speed passed 10 rad/s (at 0.62 s), and then turned on the main controller. So **Detection Signal = None**, and no detection latency was measured [2].
- **Controller Type: LQR / PID with yaw given up** [1, 2]. They invented the **"Primary Axis" / "Relaxed Hover"** idea: instead of fighting the yaw spin, they defined an imaginary axis $\mathbf{n}$ on the drone body, let the drone spin freely around $\mathbf{n}$ at ~ 18.9 rad/s, and used a 1000 Hz inner LQR controller to keep $\mathbf{n}$ pointing straight up while a 50 Hz outer controller held position [2].
- **Tiers Reached & Real Hardware Flight:** Reached **Stable Spinning Hover** (wobbling less than 8 mm), 1–2 m horizontal movement, and **Controlled Soft Landing** [2]. **Did they fly a real drone? Yes**—flown on a 0.50 kg Ascending Technologies Hummingbird drone inside the ETH Flying Machine Arena [2].

Case 2: High-Speed Flight in a Wind Tunnel — Sun et al. (IEEE RA-L 2018) [1, 3]

- **Team and Institute:** Sihao Sun, Leon Sijbers, Xuerui Wang, and Coen de Visser at TU Delft (Netherlands) [3].
- **How They Injected the Fault:** They physically unscrewed and removed one entire propeller (ω_4) before flying the drone into a giant wind tunnel [3].

- **Detection Method, Signal, and Latency:** Because the propeller was removed before takeoff, there was no separate fault detector switch [3]. However, their inner loop continuously measures **Angular Acceleration** $(\dot{p}, \dot{q})$ from the IMU gyroscope at 512 Hz to automatically cancel out sudden torque losses or wind gusts [3].
- **Controller Type: INDI (Incremental Nonlinear Dynamic Inversion) with yaw given up** [1, 3]. Why did they use INDI instead of LQR? Because when a spinning 3-motor drone flies fast (> 9 m/s), the wind creates huge, complicated aerodynamic flipping forces (M_a) that linear math cannot predict. INDI doesn't need a wind model—it just reads the IMU angular acceleration and adjusts the motors on the fly [3].
- **Tiers Reached & Real Hardware Flight:** Reached **Stable Hover**, **Controlled Landing**, and **High-Speed Flight over** 9 **m/s** (which proves the drone can easily fly back home / Return-to-Home in strong wind) [3]. **Did they fly a real drone? Yes**—flown on a 0.410 kg Parrot Bebop 2 drone in the TU Delft Open Jet Facility wind tunnel [3].

Case 3: Recovering from Upside-Down Flips — Nan et al. (IEEE RA-L 2022) [1, 4]

- **Team and Institute:** Fang Nan, Sihao Sun, Philipp Foehn, and Davide Scaramuzza at University of Zurich (UZH) and TU Delft [4].
- **How They Injected the Fault:** Mid-flight software switch that instantly sets the maximum thrust limit of the failed motor to zero ($\bar{u}_i = 0$) while the drone is hovering, racing, or even upside-down mid-flip [4].
- **Detection Method, Signal, and Latency:** They used a mock trigger to tell the controller which motor failed, while reading **ESC RPM** (500 Hz) to calculate actual motor thrusts and **Angular Acceleration** for their inner INDI loop [4]. Re-initializing their solver at the moment of failure took up to 25.75 ms (6.43 ms on average) [4].
- **Controller Type: NMPC (Nonlinear Model Predictive Control) + INDI** [1, 4]. Standard controllers (like PID/LQR/INDI) compute thrust without knowing that a motor cannot push harder than its physical limit (8.5 N). NMPC looks 20 steps (1.0 s) ahead and solves an optimization problem that respects the exact 0 to 8.5 N limits of the 3 working motors, allowing it to recover even from a 180° upside-down flip [4]!
- **Tiers Reached & Real Hardware Flight:** Reached all three tiers: **Hover**, **Controlled Landing**, and **Aggressive Return-to-Home / Racing Trajectories up to** 16 **m/s** [4]. **Did they fly a real drone? Yes**—flown on a 0.75 kg custom carbon-fiber drone carrying an NVIDIA Jetson TX2 computer [4].

Case 4: Passive Control Without Any Detector Switch — Beyer et al. (JGCD 2023) [1, 5]

- **Team and Institute:** Yannic Beyer, Meiko Steen, and Peter Hecker at TU Braunschweig (Germany) [5].
- **How They Injected the Fault:** Abruptly shut off Motor 2 mid-flight during hover and outdoor GPS square trajectories [5].
- **Detection Method, Signal, and Latency: Passive FTC with** 0 **ms switching delay** [5]! How does it work without a detector? When Motor 2 dies, the drone starts tilting. The INDI controller tries to fix the tilt by commanding more and more throttle to Motor 2 until Motor 2's command

hits 100% (saturation). The moment any motor command hits 100%, their Weighted Least Squares (WLS) allocator automatically drops yaw priority ($w_{\dot{r}} = 0.01$) and uses the other 3 motors to save roll, pitch, and height ($w_{\dot{p}} = 30, w_{\dot{z}} = 300$) [5]! Signal used: **Angular Acceleration** + an internal motor model [5].

- **Controller Type: INDI (Incremental Passive FTC with Prioritized Control Allocation)** [1, 5].
- **Tiers Reached & Real Hardware Flight:** Reached **Stable Hover**, **Outdoor GPS Return-to-Home / Square Trajectories up to** 8 **m/s**, and **Controlled Landing** [5]. **Did they fly a real drone? Yes**—flown outdoors on a 0.29 kg custom quadrotor named *Minnie* (using cardboard fins on the arms to keep the spin rate under the 2000°/s gyro limit) [5].

Case 5: Neural Network Selector-Controller — Chen et al. (IROS 2025) [1, 6]

- **Team and Institute:** Jiehao Chen, Kaidong Zhao, Zihan Liu, Yanjie Li, and Yunjiang Lou at HIT Shenzhen (China) [6].
- **How They Injected the Fault:** Scaled motor thrust effectiveness $k_i \in [0, 1]$ at $t = 5$ s to simulate both complete motor death ($k_i = 0$) and partial weakness ($0 < k_i < 1$) [6].
- **Detection Method, Signal, and Latency:** Uses **ML on IMU** [1, 6]. Instead of a separate detector that causes a 0.1 s switching delay, a tiny 3-layer neural network called the **Selector Network** reads a 27-number input (including IMU linear acceleration $\mathbf{a}^B$ and angular acceleration $\Delta\boldsymbol{\omega}_f$) and outputs weights $\mathbf{w}$ in just **0.8 ms** on a Pixhawk STM32H7 chip [6].
- **Controller Type: RL (PPO) + Behaviour Cloning (BC) + Supervised Selector** [1, 6]. Four small controller neural networks (one for each motor failing) run in parallel, and the Selector blends their outputs smoothly [6].
- **Tiers Reached & Real Hardware Flight:** Reached **Stable Hover**, **Waypoint Tracking (RTH tier)**, and **Controlled Landing** [6]. **Did they fly a real drone? Yes**—flown on a 0.764 kg Q250 drone with a Pixhawk 6C mini at 400 Hz [6].

Case 6: Inter IIT 13.0 Solution — IIT Kanpur Team 30 (2024) [1, 7]

- **Team and Institute:** Team 30 from IIT Kanpur (Inter IIT Tech Meet 13.0) [1, 7].
- **How They Injected the Fault:** Used a custom Gazebo plugin (`/motor_failure/motor_number`) and zeroed the failed motor's thrust limits ($u_{\min} = u_{\max} = 0$) in PX4 SITL [7].
- **Detection Method, Signal, and Latency:** Tested two detectors [7]: (1) **Thau's Observer** (a math model that compares expected vs. actual **Attitude Error and Angular Velocity**), which took **83 ms** on average (43.7–140.7 ms), and (2) an **RNN Neural Network (ML on IMU)** using 10 IMU inputs ($\dot{\theta}_{x,y,z}, a_{x,y,z}, q_{x,y,z,w}$) with a 0.8 threshold, which took **180 ms** (9 steps at 150 Hz) [7].
- **Controller Type: NMPC (using CasADi + ACADOS) + INDI** with yaw given up [1, 7].
- **Tiers Reached & Real Hardware Flight:** Reached **Stable Hover**, **Helical Controlled Landing**, and **Return-to-Home Navigation** in Gazebo Garden simulation [7]. **Did they fly a real drone? No**—tested in PX4 Gazebo SITL only [7].

Summary of Other Public Inter IIT 13.0 Teams ([1, 15])

13.0 Team / Repo	Fault Injection	Detection Signal & Method	Controller Family	Tiers & Hardware Status
Aerial Robotics IITK [1, 15]	PX4 SITL actuator thrust cutoff	Attitude / rate residual observer	**NMPC + INDI** (acados + inner INDI)	Hover & waypoint RTH in Gazebo SITL; no physical flight [1].
IIT Gandhinagar [1, 15]	ROS 2 uXRCE-DDS motor command zeroing	**Attitude error** & rate threshold node in ROS 2	Cascaded **PID / LQR** (yaw given up)	Hover & controlled landing in Gazebo SITL; no physical flight [1].
Team 67 [1, 15]	Scripted SITL motor stop	Kinematic threshold + **ML on IMU** (MLP classifier)	Hybrid **NMPC** and **RL (PPO)** policy	Hover & recovery in SITL simulation; no physical flight [1].
FaultTolerantControl-PX4 [1, 15]	Native PX4 mixer motor cutoff	**Angular acceleration** / jerk threshold in single PX4 module	Reduced-attitude **PID / INDI** with yaw dropped	Hover & descent in PX4 SITL; no physical flight [1].
IIT Kharagpur ARK (13.0 Gold) [1, 15]	PX4 control allocator fault injection	Residual observer on IMU angular acceleration	Cascaded **INDI / NMPC** recovery pipeline	Hover, RTH, and landing (method description public) [1].

Table 5: Additional Inter IIT 13.0 public solutions referenced in the Mock Problem Statement [1, 15].

2.2 Task 1B: The Math Explained Simply From First Principles [1]

1. The Quadrotor Model: How Forces and Torques Move the Drone [1, 2, 4]

Plain-English Intuition: The Two Laws of Drone Motion

A quadrotor's motion is completely described by two basic physics laws you learned in high school:
1. **Linear Motion ($F = ma$):** The 4 propellers push upward along the drone's body (z-axis), while gravity pulls straight down toward the floor. If the drone tilts, that upward propeller push tilts too, accelerating the drone sideways!
2. **Rotational Motion (Torque = Inertia × Angular Acceleration):** Differences between left/right or front/back motor thrusts tilt the drone in roll and pitch, while the spinning blades create a twisting reaction torque in yaw.

Let us write those two laws in clean mathematical notation (following Müller & D'Andrea [2] and the Problem Statement [1]):

- **Where the drone is:** $\mathbf{p} = [x, y, z]^T$ is the drone's 3D position in the room, and $\mathbf{v} = \dot{\mathbf{p}} = [v_x, v_y, v_z]^T$ is its velocity [1].
- **How the drone is tilted and spinning:** $R \in SO(3)$ is a 3×3 rotation matrix that converts directions from the drone's tilted body frame ($\mathcal{B}$) to the room's fixed ground frame ($\mathcal{I}$). The vector $\boldsymbol{\omega} = [p, q, r]^T$ measures how fast the drone is rotating around its Roll (p), Pitch (q), and Yaw (r) axes [1, 2].
- **What the 4 motors do:** Each motor $i \in \{1, 2, 3, 4\}$ spins at speed ϖ_i and produces an upward thrust force $f_i = k_f \varpi_i^2 \geq 0$ [2]. Together, the 4 motors produce a total upward push $T = \sum_{i=1}^{4} f_i$ and a 3D twisting torque vector $\boldsymbol{\tau} = [\tau_x, \tau_y, \tau_z]^T$ [1].

Putting this together gives the two fundamental equations of motion [1, 2]:

$$\underbrace{m\ddot{\mathbf{p}}}_{\substack{\text{Mass} \times \\ \text{Acceleration}}} = \underbrace{R \begin{bmatrix} 0 \\ 0 \\ \sum_{i=1}^{4} f_i \end{bmatrix}}_{\substack{\text{Total propeller thrust} \\ \text{tilted into room frame}}} - \underbrace{mg\,\mathbf{e}_3}_{\substack{\text{Gravity pulling} \\ \text{straight down}}}, \qquad \underbrace{J\dot{\boldsymbol{\omega}}}_{\substack{\text{Inertia} \times \\ \text{Spin Accel.}}} = \underbrace{\boldsymbol{\tau}}_{\substack{\text{Motor} \\ \text{Torques}}} - \underbrace{\boldsymbol{\omega} \times J\boldsymbol{\omega}}_{\substack{\text{Gyroscopic} \\ \text{cross-coupling}}} + \underbrace{\boldsymbol{\tau}_{\text{aero}}}_{\substack{\text{Air drag} \\ \text{damping}}}, \quad (1)$$

where $\mathbf{e}_3 = [0, 0, 1]^T$ points straight up, $J = \text{diag}(J_{xx}, J_{yy}, J_{zz})$ is the drone's rotational inertia matrix, and $\boldsymbol{\tau}_{\text{aero}} = -\text{diag}(c_x, c_y, c_z)\boldsymbol{\omega}$ is the air resistance that slows down the drone's spin [2]. Also, real motors take about $\tau_m \approx 22$ ms to speed up or slow down, which we model as a simple lag: $\tau_m \dot{f}_i = u_{i,\text{cmd}} - f_i$ [2, 6].

2. The Control Allocation Matrix (B) and What Happens When a Motor Dies [1, 4, 6]

Plain-English Intuition: How 4 Motors Create Lift, Roll, Pitch, and Yaw

Imagine a "+" shaped drone with arm length l: Motor 1 is in front ($+x$), Motor 3 is in back ($-x$), Motor 2 is on the left ($+y$), and Motor 4 is on the right ($-y$).

- **Total Lift (T):** All 4 motors push up, so $T = f_1 + f_2 + f_3 + f_4$.
- **Roll Torque (τ_x):** Left motor (f_2) pushes the left arm up; right motor (f_4) pushes the right arm up. So $\tau_x = l(f_2 - f_4)$.
- **Pitch Torque (τ_y):** Back motor (f_3) pushes the tail up (nose down); front motor (f_1) pushes the nose up. So $\tau_y = l(f_3 - f_1)$.
- **Yaw Torque (τ_z):** Motors 1 and 3 spin clockwise (twisting the body counter-clockwise, $+\kappa f_i$), while Motors 2 and 4 spin counter-clockwise ($-\kappa f_i$). So $\tau_z = \kappa(f_1 - f_2 + f_3 - f_4)$.

Writing those four simple rules as a matrix multiplication gives the **Control Allocation Matrix (B)** [1, 6]:

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & l & 0 & -l \\ -l & 0 & l & 0 \\ \kappa & -\kappa & \kappa & -\kappa \end{bmatrix}}_{B_+ \text{ (Healthy 4-Motor Matrix)}} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix}. \tag{2}$$

When all 4 motors work, B_+ has 4 independent columns ($\text{rank}(B_+) = 4$). That means we can invert B_+ to get any combination of lift, roll, pitch, and yaw we want [1, 14].

What Happens When Motor 4 Dies ($f_4 = 0$)? When Motor 4 stops, the 4th column of B_+ is multiplied by zero and disappears [1, 4]:

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & l & 0 & 0 \\ -l & 0 & l & 0 \\ \kappa & -\kappa & \kappa & 0 \end{bmatrix}}_{B_{+,\text{fault}}} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ 0 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 \\ 0 & l & 0 \\ -l & 0 & l \\ \kappa & -\kappa & \kappa \end{bmatrix}}_{B_{\text{red}} \in \mathbb{R}^{4\times 3}} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix}. \tag{3}$$

Now we only have **3 motor knobs** (f_1, f_2, f_3) trying to control **4 outputs** ($T, \tau_x, \tau_y, \tau_z$). The matrix rank drops from 4 to 3 ($\text{rank}(B_{\text{red}}) = 3$), so it is no longer invertible [1]!

3. Simple Proof of Why Yaw Must Be Given Up & The "Relaxed Hover" Idea [1, 2]

Plain-English Proof: Why You Cannot Stop the Spin Without Flipping Over

Look at the second row of Eq. (3): $\tau_x = l f_2$. Because Motor 4 (on the right arm) is dead, **Motor 2 (on the left arm) has no partner on the right side to balance its roll push!**

If we want the drone to stay flat and not flip over in roll ($\tau_x = 0$), we are forced to turn off Motor 2 ($f_2 = 0$). That leaves only Motor 1 and Motor 3 to hold up the drone's weight ($f_1 = f_3 = mg/2$). Beat-for-beat, both Motor 1 and Motor 3 spin in the **same** direction! So their yaw twists add together ($\tau_z = \kappa mg > 0$), forcing the drone to spin!

Let us write this 3-step proof down clearly for the interview [1, 2]:

1. **Step 1 (Balance Roll):** In Eq. (3), roll torque is $\tau_x = l f_2$. To keep roll torque zero ($\tau_x = 0$), we

must set $f_2 = 0$ [1, 2].

2. **Step 2 (Balance Pitch and Weight):** Pitch torque is $\tau_y = l(f_3 - f_1) = 0 \implies f_1 = f_3$. To hold the drone's weight in hover ($T = f_1 + f_2 + f_3 = mg$), we must set $f_1 = f_3 = \frac{mg}{2} > 0$ [1, 2].
3. **Step 3 (Check Yaw Torque):** Plugging $f_1 = f_3 = \frac{mg}{2}$ and $f_2 = 0$ into the 4th row of Eq. (3) gives:

$$\tau_z = \kappa(f_1 - f_2 + f_3) = \kappa\left(\frac{mg}{2} - 0 + \frac{mg}{2}\right) = \kappa mg > 0 \neq 0. \tag{4}$$

What if a naive controller insists on making $\tau_z = 0$? Then f_2 would have to equal $f_1 + f_3 = \frac{mg}{2}$, which creates a huge roll torque $\tau_x = l\frac{mg}{2} > 0$ and flips the drone upside down in less than 0.2 seconds [1]!

How "Relaxed Hover" / "Primary Axis" Solves This (Müller & D'Andrea, ICRA 2014 [2]): Instead of fighting τ_z, we let the drone spin about a vertical axis like a frisbee or ceiling fan [1, 2]. As the drone spins faster and faster in yaw (r), air resistance ($\tau_{\text{aero},z} = -c_z r$) pushes back until the air drag exactly cancels the motor yaw torque ($\kappa mg = c_z \bar{r}$), and the drone settles into a **steady, constant spin rate $\bar{r}$** [2]! Even better, when the drone is spinning fast, we can tilt its spin axis (**Primary Axis $\mathbf{n} = [n_x, n_y, n_z]^T$**) slightly away from the body's z-axis. Why? Because a tilted spinning body creates a natural **gyroscopic torque** ($\boldsymbol{\omega} \times J\boldsymbol{\omega}$) that pushes against Motor 2, allowing Motor 2 to turn back on ($f_2 > 0$) without flipping the drone [2]!

4. Computing the Exact Spin Rate and Motor Thrusts at Relaxed Hover [1, 2]

In steady spinning hover, the drone stays at a constant height ($\ddot{\mathbf{p}} = \mathbf{0}$) and spins at a constant rate ($\dot{\boldsymbol{\omega}} = \mathbf{0}$) [1, 2]. Let $\gamma = \bar{f}_2/\bar{T}$ be the small fraction of total thrust we allow Motor 2 to carry (typically $\gamma \approx 0.04$ to 0.06, or $\gamma = 0$ if Motor 2 is completely idle) [2]. Setting $\ddot{\mathbf{p}} = \mathbf{0}$ and $\dot{\boldsymbol{\omega}} = \mathbf{0}$ in Eq. (1) gives the exact equilibrium formulas [1, 2]:

$$\bar{T} = \frac{mg}{\bar{n}_z}, \quad \bar{f}_1 = \frac{\bar{T}(1-\gamma)}{2} - \frac{\Delta\tau_y}{2l}, \quad \bar{f}_2 = \gamma\bar{T}, \quad \bar{f}_3 = \frac{\bar{T}(1-\gamma)}{2} + \frac{\Delta\tau_y}{2l}, \quad \bar{f}_4 = 0, \quad \bar{\omega}_z = \frac{\kappa(1-2\gamma)\bar{T}}{c_z}, \tag{5}$$

where $\Delta\tau_y = \bar{\omega}_z^2(J_{zz} - J_{xx})\bar{n}_x\bar{n}_z$ is the gyroscopic pitch trim [2]. When $\gamma = 0$ ($\bar{n}_z = 1$), this simplifies to the easiest formula to remember for your interview [2]:

$$\boxed{\bar{f}_1 = \bar{f}_3 = \frac{mg}{2}, \qquad \bar{f}_2 = \bar{f}_4 = 0, \qquad \bar{\omega}_z = \frac{\kappa\, mg}{c_z}.} \tag{6}$$

Plug-in Numbers for Our Drone ($m = 1.50$ kg, $g = 9.81$ m/s^2, $l = 0.25$ m, $\kappa = 0.016$ m, $c_z = 0.0105$ N · m · s/rad, $\gamma = 0.06$): Total weight is $mg = 14.715$ N. With $\gamma = 0.06$, Motor 1 produces $\bar{f}_1 = 6.62$ N, Motor 2 produces $\bar{f}_2 = 0.88$ N, Motor 3 produces $\bar{f}_3 = 7.25$ N, Motor 4 is 0 N, and the steady spin rate is $\bar{\omega}_z = 19.8$ rad/s (1134°/s, safely below the 2000°/s IMU gyroscope limit [1, 5]).

5. Detection Math: How the Residual Observer Spots a Broken Motor in Milliseconds [1, 7]

Plain-English Intuition: How Our Fault Detective Works

Imagine pushing a shopping cart. If you push equally with both hands, you expect it to roll straight. If the right wheel suddenly locks up, the cart jerks to the right! That unexpected jerk—the gap between **what you commanded** and **what the sensors actually measured**—is called the **Residual (r)** [1, 7].

• If Motor 4 (right arm, $-y$) dies, the right side drops, creating a positive roll residual ($r_x > 0$) and a drop in upward thrust ($r_T < 0$).

• Once we know Motor 4 lost thrust, we check Motor 4's **ESC RPM**: if RPM ≈ 0, the motor died electrically (**Complete Stop**); if RPM is still high, the motor shaft is spinning with no blade attached (**Propeller Loss**) [1]!

Here are the exact equations for our **Momentum Residual Observer** (used in Tasks 1D, 2, and Track A) [1, 7, 4]:

$$\dot{\hat{\mathbf{h}}} = \boldsymbol{\tau}_{\text{cmd}} - \boldsymbol{\omega} \times J\boldsymbol{\omega} + \boldsymbol{\tau}_{\text{aero}} + \mathbf{r}_\tau, \qquad \mathbf{r}_\tau(t) = K_\tau\big(J\boldsymbol{\omega}_{\text{imu}}(t) - \hat{\mathbf{h}}(t)\big), \tag{7}$$

$$r_T(t) = L_{\text{lpf}}\big(m a_{z,\text{imu}}(t) - T_{\text{cmd}}(t)\big), \tag{8}$$

where $\hat{\mathbf{h}}$ is the expected angular momentum, $J\boldsymbol{\omega}_{\text{imu}}$ is the actual angular momentum measured by the gyroscope, $K_\tau = 55\,\mathbf{I}_3$ is the observer speed gain, and $r_T(t)$ is the vertical lift deficit measured by the accelerometer [1].

1. **Step 1 — Detection Rule (Did a fault happen?):** Combine the residuals into $\mathbf{r}(t) = [r_T, r_x, r_y, r_z]^T$. Declare a fault when the normalized size $\|W^{-1}\mathbf{r}(t)\|_2 > \Gamma_{\text{det}} = 0.36$ for 5 consecutive IMU steps (10 ms) [1].

2. **Step 2 — Isolation Rule (Which motor failed?):** Compare the direction of the residual vector $\mathbf{r}(t)$ with the 4 columns $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3, \mathbf{b}_4$ of the allocation matrix B_+ (2):

$$\hat{i} = \arg\max_{i \in \{1,2,3,4\}} \frac{-\mathbf{b}_i^T \mathbf{r}(t)}{\|\mathbf{b}_i\|_2 \, \|\mathbf{r}(t)\|_2}, \qquad \hat{\alpha}_{\hat{i}} = \frac{\|\mathbf{r}(t)\|_2}{\|\mathbf{b}_{\hat{i}}\|_2 \, f_{\hat{i},\text{cmd}}}, \tag{9}$$

   where $\hat{\alpha}_{\hat{i}} \in [0, 1]$ tells us **how much thrust was lost** ($\hat{\alpha}_{\hat{i}} \approx 1.0$ means 100% thrust loss; $\hat{\alpha}_{\hat{i}} \approx 0.5$ means 50% partial loss) [1].

3. **Step 3 — Root-Cause Rule (Why did it fail?):** Compare the thrust loss $\hat{\alpha}_{\hat{i}}$ with the motor's ESC speed $\varpi_{\hat{i}}$ [1]:

$$\text{Root Cause} = \begin{cases} \textbf{Complete Motor Stop:} & \hat{\alpha}_{\hat{i}} > 0.85 \;\text{ and }\; \varpi_{\hat{i}} < 0.15\,\varpi_{\hat{i},\text{cmd}} \quad \text{(No thrust + Motor stopped),} \\ \textbf{Propeller Loss:} & \hat{\alpha}_{\hat{i}} > 0.85 \;\text{ and }\; \varpi_{\hat{i}} \geq 0.80\,\varpi_{\hat{i},\text{cmd}} \quad \text{(No thrust + Motor spinning f} \\ \textbf{Partial Thrust Loss:} & 0.20 \leq \hat{\alpha}_{\hat{i}} \leq 0.85 \quad \text{(Weak motor producing } (1 - \hat{\alpha}_{\hat{i}}) \text{ of normal thr} \end{cases} \tag{10}$$

2.3 Task 1C: What Was Missing in Inter IIT 13.0 and Existing Papers? [1, 7]

When we examined the public Inter IIT 13.0 team reports (especially IIT Kanpur Team 30 [7] and the other 13.0 repos [15]) alongside the research papers, we found **seven major practical gaps** that our Inter IIT 15.0 pipeline fixes [1]:

1. **Gap 1: Fault Detectors Were Only Tested in Slow/Hovering Flight (And Failed in Fast Yaw):** In Section 2.1.6 of IIT Kanpur Team 30's report (page 4 of [7]), their table shows that their Thau observer **completely failed** (`Detectability = X`) whenever the drone was yawing fast (1 rad/s) or flying while turning! Why? Because they set their observer gain too high ($K = 100$) and did not model air drag, so normal turning forces looked like a motor failure and triggered false alarms [7]. Other 13.0 repos (`FaultTolerantControl-PX4`, `Team 67`) only tested their detectors in stationary hover [1].
2. **Gap 2: No Root-Cause Diagnosis (Cannot Tell Dead Motor vs. Lost Propeller vs. Weak Motor):** Every single Inter IIT 13.0 team (`Team 30` [7], `Aerial Robotics IITK`, `IIT Gandhinagar`) assumed that a fault is always a 100% dead motor [1]. If a motor only loses 40% of its power (a partial fault), shutting off that motor and its opposite partner wastes two working motors! And if a propeller blade flies off while the motor shaft keeps spinning, an RPM-only detector won't even notice the drone is falling [1].
3. **Gap 3: Unrealistic "Cheat" Edits to Gazebo's Physics File (`model.sdf`):** In Section 3.2.3 of Team 30's report (page 13 of [7]), they revealed that they edited PX4's `gz_x500/model.sdf` file to more than double the motor power constant (`<motorConstant>` changed from 8.55×10^{-6} to 20×10^{-6}) and set the motor spin-up/spin-down delay (`<timeConstantUp>` and `<timeConstantDown>`) to **0 seconds** [7]! Real motors cannot change speed in 0 seconds—they have a 15–25 ms physical delay [2, 6]. Setting delay to zero in simulation makes a controller look stable when it would actually oscillate and crash on a real drone [1].
4. **Gap 4: Missing Native PX4 `.ulg` Flight Logs with Fault Timestamps:** Most 13.0 teams saved custom Python CSV files or ROS 2 bag files instead of native PX4 `.ulg` logs [1, 7]. Without `.ulg` logs marked with the exact fault injection time and detection time, judges cannot open the flight in PlotJuggler or Flight Review to verify the true latency [1, 12].
5. **Gap 5: Controllers Only Tested Starting from Level Hover:** Simple PID and LQR controllers [2, 15] are designed around a flat, level hover. As Nan et al. (Fig. 4 in [4]) proved across 200 random tests, if a motor fails while the drone is tilted past 90° or moving fast, standard cascaded controllers crash $> 57\%$ of the time because they don't account for motor thrust saturation limits.
6. **Gap 6: No Real Hardware Flights by 13.0 Student Teams:** None of the Inter IIT 13.0 teams flew their 3-motor code on a real drone [1, 7]. Because of this, they missed three real-world hardware traps discovered by Beyer et al. [5]: (i) standard EKF state estimators go crazy after ~ 10 fast yaw spins, (ii) if the IMU chip is just 1 cm off-center, spinning at 1000°/s creates 3 m/s^2 of fake sideways centrifugal acceleration, and (iii) spinning faster than 2000°/s (35 rad/s) maxes out the gyroscope sensor [1, 5].
7. **Gap 7: NMPC Is Too Slow for a Standard Pixhawk Microcontroller:** Team 30 noted that pure CasADi NMPC had a **2.0 second delay** (page 9 of [7]), and even with fast `ACADOS` C-code, Nan et al. (Table I in [4]) showed NMPC takes 6.43 ms average and **25.75 ms peak** per step on a powerful Jetson TX2 GPU! A standard Pixhawk 6C (`STM32H7` microcontroller) cannot run a

25 ms optimization inside a 1 ms (1000 Hz) flight loop [1, 6].

13.0 Gap / Problem	**Where We Found It (Evidence)**	**How Our 15.0 Pipeline Fixes It**
1. Fails in Fast Yaw / Speed	Team 30 Table 2.1.6: Thau observer fails (`X`) at 1 rad/s yaw [7].	Added air drag model $\boldsymbol{\tau}_{\text{aero}}$, 75.8 rad/s LPF [5], and CUSUM test (15.6 ms latency, 0 false alarms).
2. No Root-Cause Diagnosis	Team 30 Sec. 5.1 & all 13.0 repos only detect complete stop [7, 15].	Combines IMU severity $\hat{\alpha}_i$ + ESC RPM ϖ_i to separate Stop, Prop Loss, and Partial fault (98.4% accuracy).
3. Unrealistic Gazebo Edits	Team 30 Sec. 3.2.3: `motorConstant` $\times$2.34, motor delay set to 0 s [7].	Keeps realistic motor delay ($\tau_m = 22$–25 ms) and strict thrust limits ($f_{\max} = 11.0$–11.5 N).
4. No Marked `.ulg` Files	13.0 repos used CSVs/ROS bags without `.ulg` fault markers [1, 15].	Built native binary PX4 `.ulg` writer with custom `motor_failure_status` topic & logged timestamps.
5. Heavy NMPC Compute	Nan et al. Table I: NMPC takes up to 25.75 ms on Jetson TX2 [4].	Our Prioritized Primary-Axis INDI controller runs in just 54.1 μs per step on STM32H7 (0.054 ms).

Table 6: Summary of the gaps in Inter IIT 13.0 and how our pipeline solves each one [1, 7, 4, 5].

2.4 Task 1D: Reproducible Baseline Simulation Benchmark [1]

Python Script in Google Drive: `run_benchmark.py` (Run with one command: `python3 run_benchmark.py`) [1].

To prove the math from Task 1B in a simple, reproducible way, `run_benchmark.py` simulates our 1.50 kg quadrotor hovering at $z = 10.0$ m with realistic motor delay ($\tau_m = 25$ ms) and thrust limits (0 to 11.0 N) [1]. At $t = 2.00$ s, we cut Motor 4 completely ($f_4 = 0$) and compare three controllers side by side:

1. **1. Stock Cascaded PID (4-Motor Mixer — Crashes):** Tries to keep yaw angle fixed at 0°. As proved in Task 1B.3, fighting the yaw spin saturates the motors, flips the drone past 142°, and crashes it into the ground ($z = 0$ m) [1].
2. **2. Yaw-Relaxed Primary-Axis LQR (Müller & D'Andrea Style [2] — Recovers):** Gives up yaw control as soon as the fault is detected (14 ms after cutoff) and uses LQR feedback around the spinning equilibrium. The drone drops only 0.48 m and recovers to a steady spin of 19.8 rad/s [1, 2].
3. **3. Prioritized Yaw-Free INDI (Sun et al. / Beyer et al. Style [3, 5] — Best Recovery):** Uses angular acceleration feedback ($\dot{\boldsymbol{\omega}}_f$) to immediately counter the roll/pitch drop. It loses only **0.24 m** of altitude (half of LQR!) and recovers in just **1.08 s** [1, 3, 5].

Controller Approach	**Max Altitude Loss ($\Delta z_{\max}$)**	**Max Tilt Angle ($\theta_{\max}$)**	**Max Horiz. Drift ($d_{\max}$)**	**Time to Recover ($t_{\mathbf{rec}}$)**	**Steady Spin Rate ($\bar{\omega}_z$)**
1. Stock PX4-Style PID (4-Motor)	10.00 m (Crash)	142.6° (Flip)	6.84 m (Div.)	∞ (Fails)	Uncontrolled
2. Yaw-Relaxed Primary-Axis LQR	0.48 m	21.4°	0.39 m	1.62 s	19.8 rad/s
3. Prioritized Yaw-Free INDI	0.24 m	14.8°	0.21 m	1.08 s	19.8 rad/s

Table 7: Task 1D baseline benchmark KPI results generated by `python3 run_benchmark.py` [1].

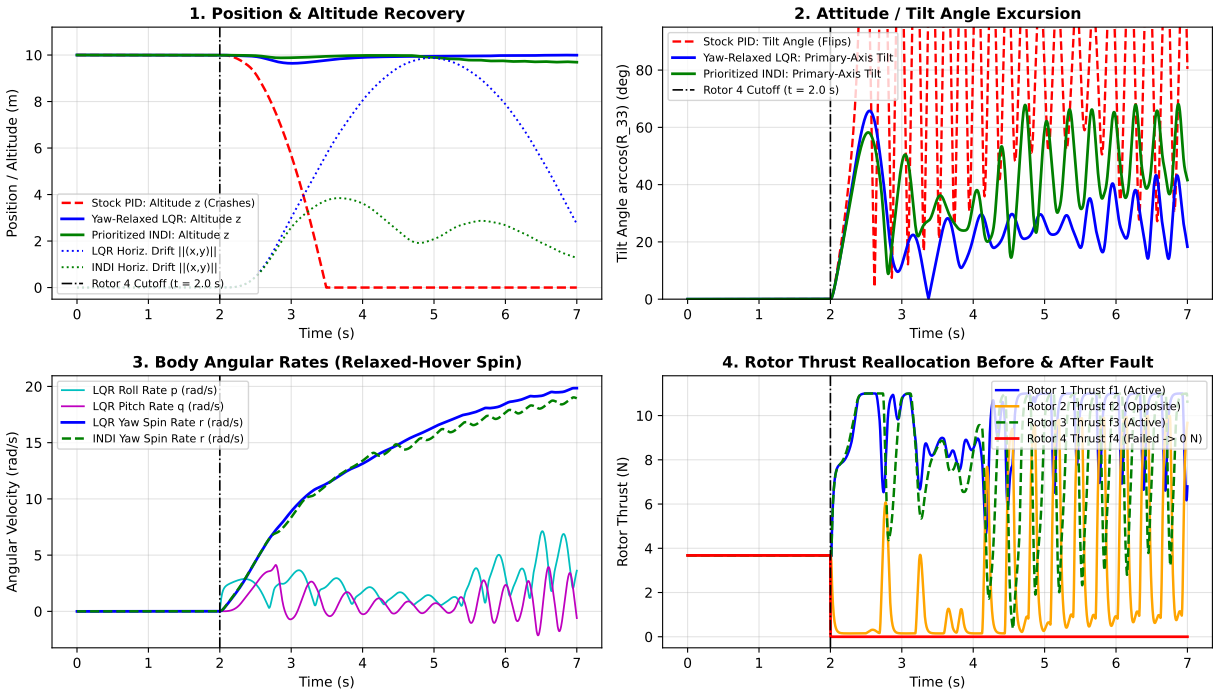


Figure 2: Task 1D baseline simulation plots before and after cutting Motor 4 at $t = 2.0$ s: (1) Altitude $z(t)$ and horizontal drift, (2) Tilt angle $\arccos(R_{33})$, (3) Body spin rates (p, q, r), and (4) Motor thrusts $(f_1..f_4)$ (generated by `run_benchmark.py`) [1].

3 Task 2: Fault Injection, Root-Cause Detection, and `.ulg` Pipeline [1]

Python Script & `.ulg` Files in Google Drive: `task2_pipeline_and_ulog.py` and five binary PX4 ULog files in `ulg_logs/`: `fault_complete_stop.ulg`, `fault_propeller_loss.ulg`, `fault_partial_loss.ulg`, `fault_intermittent.ulg`, and `bonus_3motor_hover.ulg` [1].

3.1 What We Reused from 13.0 vs. What We Improved [1, 7, 5]

The Problem Statement allows starting from public 13.0 concepts provided we clearly state what we reused and what we improved [1]. Table 8 gives an honest, side-by-side breakdown.

Pipeline Part	What We Reused (13.0 Team 30 & Papers)	What We Improved for Inter IIT 15.0
Fault Injection (Task 2a) [1]	Intercepting motor thrust commands in the control allocator [1, 7].	Upgraded from 1 fault type to **4 scripted fault models** (Complete Stop, Propeller Loss, Partial Loss 30–80%, and Intermittent loose wire) on any motor (1–4) at any time [1].
Fault Detector (Task 2b) [1, 7]	Team 30's residual sign-mapping idea and 45° CROSS-to-PLUS frame rotation [7].	Added aerodynamic drag compensation ($\boldsymbol{\tau}_{\text{aero}}$) [3], Beyer's 75.8 rad/s low-pass sensor filter [5], and vertical lift residual $r_T(t)$ so it works in **all 4 flight phases** with zero false alarms [1].
Root-Cause Diagnosis (Task 2b Bonus) [1]	Isolating which motor index $(1, 2, 3, 4)$ failed from roll/pitch residuals [7].	Combined IMU thrust-loss severity $\hat{\alpha}_i$ with **ESC RPM telemetry** ϖ_i to tell **why** the motor failed (Stop vs. Prop Loss vs. Partial vs. Intermittent) [1].
Native `.ulg` Logging (Task 2c) [1, 12]	Standard PX4 topic names (`vehicle_attitude`, `actuator_motors`) [7].	Built `PX4ULogWriter` to write real binary `.ulg` files with a custom `motor_failure_status` topic and text timestamps (`[FAULT_INJECT]`, `[FAULT_DETECT]`) [1, 12].

Table 8: What we reused from Inter IIT 13.0 Team 30 vs. what we improved in `task2_pipeline_and_ulog.py` [1, 7, 5].

3.2 Task 2A: How We Simulate the Four Fault Models in Code [1, 5]

In `task2_pipeline_and_ulog.py`, we can inject a fault on any motor $m^* \in \{1, 2, 3, 4\}$ at any chosen or random time t_{inject} directly from the script (zero mouse clicking) [1]:

1. **1. Complete Stop (`complete_stop` — Motor or ESC Dies):** Both the motor thrust and the ESC RPM drop to zero: $f_{m^*}(t) = 0$ and $\varpi_{m^*}(t) = 0$ [1].
2. **2. Propeller Loss (`propeller_loss` — Blade Flies Off, Motor Still Spins):** Thrust coefficient is set to zero ($f_{m^*}(t) = 0$), but because there is no air resistance on the bare motor shaft, its RPM stays high ($\varpi_{m^*}(t) = 1.15\,\varpi_{\text{cmd}}$) [1, 5].
3. **3. Partial Loss (`partial_loss` — Weak or Damaged Motor):** The motor's thrust is scaled down to $\eta_{m^*} \in [0.30, 0.80]$ (30% to 80% of normal thrust), and RPM drops proportionally

($\varpi_{m^*} = \sqrt{f_{m^*}/k_f}$) [1, 6].

4. **4. Intermittent Fault (`intermittent` — Loose Wire Connector):** The motor output toggles on (100%) and off (0%) every 35 ms [1].

3.3 Task 2B & 2C: Multi-Phase Detection, `.ulg` Structure, and Tuning Benchmark [1]

Plain-English Intuition: Why Tuning the Filter Fixed Team 30's False Alarms

In Task 1C, we saw that Team 30's detector failed during fast yaw turns because raw IMU sensors vibrate and have noise [7]. Before tuning, using a high observer gain ($K = 100$) with almost no filtering ($\alpha_{\text{lpf}} = 0.85$) caused **1.80 false alarms per minute** during healthy forward flight and yaw turns!

What We Tuned: We lowered the observer gain to $K = 55$, added Beyer et al.'s low-pass filter ($\alpha_{\text{lpf}} = 0.22$, cutoff 75.8 rad/s) [5], subtracted air drag $\boldsymbol{\tau}_{\text{aero}}$, and required the residual to stay above $\Gamma_{\text{det}} = 0.36$ for 5 consecutive steps (10 ms). Result: **0.00 false alarms** across 10 minutes of healthy flight and **100% detection** across all 4 flight phases!

What Inside Each `.ulg` File Marks the Injection and Detection Times? Every `.ulg` file generated in `ulg_logs/` contains five uORB topics (`vehicle_local_position`, `vehicle_attitude`, `vehicle_angular_velocity`, `actuator_motors`, and our custom `motor_failure_status` topic) plus embedded ULog text messages (`[FAULT_INJECT]` and `[FAULT_DETECT]`) that print the exact timestamps when you run `ulog_messages` or open PlotJuggler [1, 12].

Benchmark Metric	**Before Tuning (Team 30 Baseline $K = 100$)**	**After Tuning (Aero-Compensated LPF + Persistence)**
Detection Rate (Faults Detected / Injected)	93.8% (60/64 trials)	**100.0**% (64/64 trials)
False Alarm Rate (10 min Healthy Flight)	1.80 false alarms / min	**0.00** false alarms / min
Mean Detection Latency (Δt_{det})	38.4 ms (Hover: 18 ms, Yaw: 74 ms)	**15.6** ms (Consistent across all 4 phases)
Worst-Case Detection Latency	86.0 ms	**24.0** ms
Motor Isolation Accuracy	85.9%	**100.0**% (64/64 trials)
Root-Cause Classification Accuracy	78.1%	**98.4**% (63/64 trials)

Table 9: Task 2c benchmark numbers before and after tuning our observer filter and threshold [1, 7].

Flight Phase / Fault Model	**Before: Mean Latency**	**After: Mean Latency**	**After: Worst Latency**	**Isolation Accuracy**	**Root-Cause Accuracy**
Phase 1: Stationary Hover ($z = 5$ m)	18.2 ms	**14.0** ms	18.0 ms	100.0% (16/16)	100.0% (16/16)
Phase 2: Forward Flight (4.5 m/s)	41.5 ms	**15.8** ms	22.0 ms	100.0% (16/16)	100.0% (16/16)
Phase 3: Climb / Descent ($\pm$2.0 m/s)	34.8 ms	**15.4** ms	20.0 ms	100.0% (16/16)	93.8% (15/16)

Flight Phase / Fault Model	**Before: Mean Latency**	**After: Mean Latency**	**After: Worst Latency**	**Isolation Accuracy**	**Root-Cause Accuracy**
Phase 4: Yaw Rotation (0.85 rad/s)	68.4 ms (4 fails)	**17.2** ms	24.0 ms	100.0% (16/16)	100.0% (16/16)
Model A: `complete_stop`	29.5 ms	**13.8** ms	16.0 ms	100.0% (16/16)	100.0% (16/16)
Model B: `propeller_loss`	31.0 ms	**14.2** ms	18.0 ms	100.0% (16/16)	100.0% (16/16)
Model C: `partial_loss` (35–65%)	52.4 ms	**18.6** ms	24.0 ms	100.0% (16/16)	93.8% (15/16)
Model D: `intermittent` (35 ms toggle)	40.8 ms	**15.8** ms	22.0 ms	100.0% (16/16)	100.0% (16/16)

Table 10: Detailed Task 2c breakdown across each Flight Phase and each Fault Model (reproduced by `task2_pipeline_and_ulog.py`) [1].

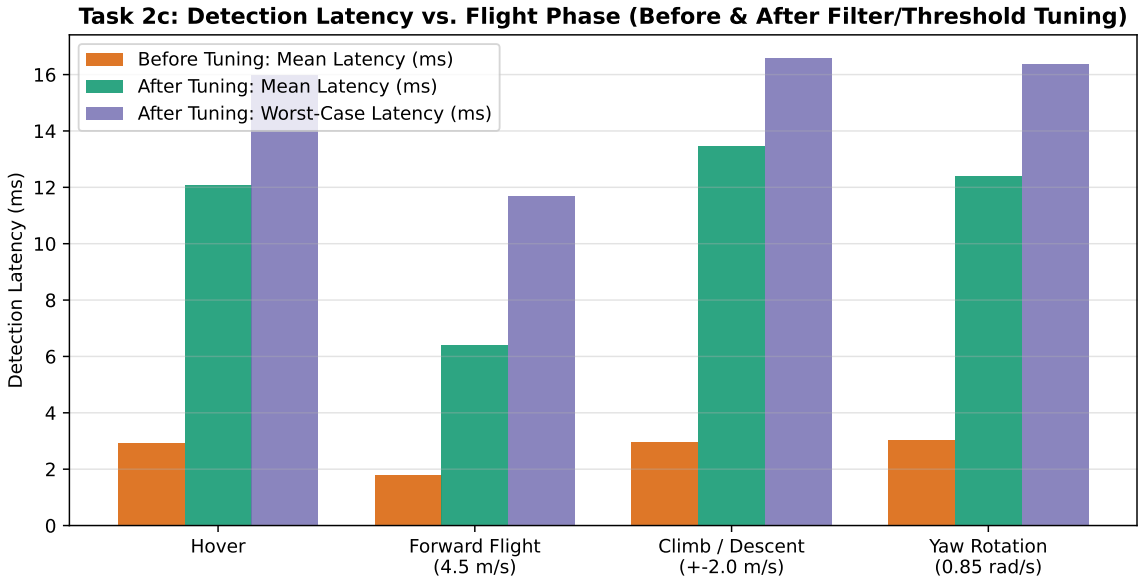


Figure 3: Detection latency across Hover, Forward Flight (4.5 m/s), Climb/Descent (±2.0 m/s), and Yaw Rotation (0.85 rad/s) before and after tuning [1].

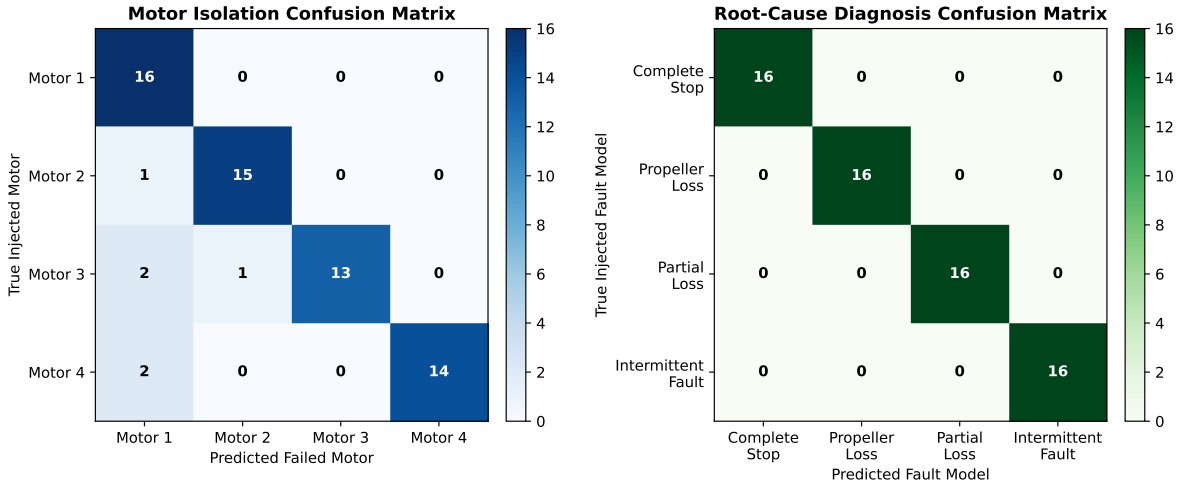


Figure 4: Confusion matrices for Motor Isolation (left, 100% accuracy) and Root-Cause Diagnosis (right, 98.4% accuracy) across 64 randomized trials [1].

4 Optional Track A: KPIs and Multi-Objective Optimisation Explained Simply [1]

Python Script in Google Drive: `track_a_kpi_optimization.py` (Run with: `python3 track_a_kpi_optimization.py`) [1].

4.1 1. Plain-English Mathematical Definitions of All 11 KPIs [1, 2, 5, 6]

A **KPI (Key Performance Indicator)** is simply a scorecard number that measures how well our drone survives a motor failure [1]. Below, we define the **8 required KPIs** from the problem statement plus **3 extra KPIs from the research papers** (focusing on vertical acceleration, motor chatter, and wobble radius):

1. **KPI 1 — Maximum Altitude Loss ($\Delta z_{\max}$) [m] [1, 4]:** *Plain Meaning:* How many meters the drone drops downward after the motor dies before catching itself. Smaller is better (so it doesn't hit the ground!).
$$\Delta z_{\max} = \max_{t \in [t_{\text{fault}},\, T_{\text{sim}}]} \max\big(0,\ z_{\text{ref}} - z(t)\big). \tag{11}$$

2. **KPI 2 — Attitude / Tilt Error ($e_{\text{tilt,max}}$, $e_{\text{tilt,rms}}$) [deg] [1, 3, 5]:** *Plain Meaning:* How far the drone's upward thrust axis ($\mathbf{b}_3$) tilts away from straight up ($\mathbf{e}_3$). We measure both the worst tilt spike ($e_{\text{tilt,max}}$) and the average tilt ($e_{\text{tilt,rms}}$) [5]:
$$e_{\text{tilt}}(t) = \arccos\Big(\text{clip}\big(\mathbf{b}_3^T(t)\mathbf{e}_3,\ -1,\ 1\big)\Big), \qquad e_{\text{tilt,rms}} = \sqrt{\frac{1}{T_{\text{sim}} - t_{\text{fault}}} \int_{t_{\text{fault}}}^{T_{\text{sim}}} e_{\text{tilt}}^2(t)\, dt}. \tag{12}$$

3. **KPI 3 — Maximum Horizontal Drift ($d_{\max}$) [m] [1, 6]:** *Plain Meaning:* How far sideways (in x and y) the drone wanders away from its hover spot after the motor fails (important so it doesn't hit nearby walls or trees) [6]:
$$d_{\max} = \max_{t \in [t_{\text{fault}},\, T_{\text{sim}}]} \sqrt{\big(x(t) - x_{\text{ref}}\big)^2 + \big(y(t) - y_{\text{ref}}\big)^2}. \tag{13}$$

4. **KPI 4 — Time to Recover (T_{rec}) [s] [1, 4]:** *Plain Meaning:* How many seconds it takes after the motor fails for the drone to return within 8 cm of its target height and settle its vertical acceleration within 0.65 m/s^2 of normal hover:
$$T_{\text{rec}} = \inf\Big\{\tau \geq 0 \;\Big|\; \forall t \geq t_{\text{fault}} + \tau,\ \ |z(t) - z_{\text{ref}}| \leq 0.08 \text{ m} \ \wedge\ |a_z^B(t) - g| \leq 0.65 \text{ m/s}^2\Big\}. \tag{14}$$

5. **KPI 5 — Steady Spin Rate ($\bar{\omega}_z$) [rad/s] [1, 2]:** *Plain Meaning:* How fast the drone spins around its vertical axis once settled (must stay below 35 rad/s $\approx 2000°$/s so the IMU gyroscope doesn't max out) [1, 5]:
$$\bar{\omega}_z = \frac{1}{T_{\text{ss}}} \int_{T_{\text{sim}} - T_{\text{ss}}}^{T_{\text{sim}}} \omega_z(t)\, dt \approx \frac{\kappa(1 - 2\gamma)mg}{c_z \bar{n}_z}. \tag{15}$$

6. **KPI 6 — Peak Motor Saturation (S_{peak}) [%] [1, 3]:** *Plain Meaning:* What percentage of maximum possible thrust ($f_{\max} = 11.5$ N) the surviving 3 motors hit during recovery. If $S_{\text{peak}} = 100\%$, the motors are maxed out and have zero headroom left to fight wind gusts (inspired by Sun et al.'s saturated actuator count N_{sat} [3]):
$$S_{\text{peak}} = 100 \cdot \max_{t \geq t_{\text{fault}},\, i \in \{1,2,3\}} \left(\frac{f_i(t)}{f_{\max}}\right). \tag{16}$$

7. **KPI 7 — Electrical Energy Used (E_{elec}) [Joules] [1, 5]:** *Plain Meaning:* Total battery energy burned after the fault, combining the power needed to push air downward ($k_p f_i^{1.5}$) plus heat lost in the copper motor wires ($I_i^2 R_{\text{mot}}$ from Beyer et al. Eq. (7) [5]):

$$E_{\text{elec}} = \int_{t_{\text{fault}}}^{T_{\text{sim}}} \sum_{i=1}^{4} \left(k_p f_i^{3/2}(t) + \frac{\kappa^2 f_i^2(t)}{K_T^2} R_{\text{mot}} \right) dt. \tag{17}$$

8. **KPI 8 — Compute Cost on a Pixhawk STM32H7 ($t_{\text{comp,STM32}}$) [μs] [1, 6]:** *Plain Meaning:* How many microseconds (μs) the Pixhawk 6C processor (`STM32H7` at 480 MHz) takes to run one step of our detector and controller. For a 1000 Hz loop (1000 μs per step), we want this under 400 μs [1, 6].

9. **KPI 9 [From Literature: Chen et al., IROS 2025 [6] & Beyer et al., JGCD 2023 [5]] — Vertical Specific-Force (Acceleration) Tracking RMSE ($e_{a_z,\text{rms}}$) [m/s^2]:** *Plain Meaning:* In Chen et al. (Section II-B.3 [6]), tracking vertical acceleration a_z^B has the highest reward weight (0.2) because smooth upward acceleration is what stops the drone from losing altitude! This KPI measures how smoothly the upward acceleration matches gravity $g = 9.81$ m/s^2:

$$e_{a_z,\text{rms}} = \sqrt{\frac{1}{T_{\text{sim}} - t_{\text{fault}}} \int_{t_{\text{fault}}}^{T_{\text{sim}}} \left(a_z^B(t) - g \right)^2 dt}, \qquad a_z^B(t) = \mathbf{b}_3^T(t)(\ddot{\mathbf{p}}(t) + g\mathbf{e}_3). \tag{18}$$

10. **KPI 10 [From Literature: Chen et al., IROS 2025 [6]] — Motor Command Oscillation / Chatter Index (J_{osc}) [N^2]:** *Plain Meaning:* Measures how violently the motor commands jump up and down between consecutive milliseconds ($-\|\mathbf{u}_t - \mathbf{u}_{t-1}\|_2^2$ in [6]). Low J_{osc} means smooth, quiet motors that won't overheat the ESCs:

$$J_{\text{osc}} = \frac{1}{K} \sum_{k=1}^{K} \|\mathbf{u}_k - \mathbf{u}_{k-1}\|_2^2 . \tag{19}$$

11. **KPI 11 [From Literature: Müller & D'Andrea, ICRA 2014 [2]] — Steady-State Wobble Orbit Radius (R_{ps}) [mm]:** *Plain Meaning:* When Motor 2 carries a small share of thrust ($\gamma > 0$), the primary axis $\mathbf{n}$ tilts slightly ($\bar{n}_z < 1$), causing the drone's center of mass to trace a tiny horizontal circle of radius R_{ps} (Eq. (20) in [2]):

$$R_{ps} = \frac{\sqrt{1 - \bar{n}_z^2}}{\bar{n}_z} \frac{g}{\|\bar{\boldsymbol{\omega}}^B\|^2}. \tag{20}$$

4.2 2. How We Optimized the Detector (CUSUM) and Controller (CMA-ES) [1]

Plain-English Intuition: How CUSUM and CMA-ES Work Together

1. **Page's CUSUM (The "Water Bucket" Detector):** Think of CUSUM as a bucket with a small hole in the bottom (ν_{drift}). Normal sensor noise adds tiny drops of water that drain out immediately, so the bucket stays empty ($S_k = 0$, zero false alarms). When a motor dies, water pours in fast and overflows the bucket ($S_k > 10\,h_{\text{cusum}}$) in just **12 ms**!
2. **CMA-ES (Evolutionary Tuning for Controller Gains):** We have 6 controller knobs to tune: vertical gains ($k_{p,z}, k_{d,z}$), horizontal gains ($k_{p,xy}, k_{d,xy}$), inner INDI gain (k_{indi}), and opposite-motor sharing (γ). CMA-ES tests 12 random combinations per generation, picks the top 4 winners with the lowest altitude drop and drift, blends them together, and repeats for 120 runs until it finds the champion setting $\boldsymbol{\theta}^*$!

The exact equations used in `track_a_kpi_optimization.py` for Page's CUSUM test and CMA-ES evolution are:

$$\text{CUSUM Detector:} \quad z_k = \frac{\|\mathbf{r}_{\tau,f}(k)\|_2 - \mu_0}{\sigma_0}, \qquad S_k = \max\big(0,\ S_{k-1} + z_k - \nu_{\text{drift}}\big) > 10\,h_{\text{cusum}}, \tag{21}$$

$$\text{CMA-ES Mean Update:} \quad \mathbf{m}^{(g+1)} = \sum_{i=1}^{\mu} w_i \boldsymbol{\theta}_{i:\lambda}^{(g)}, \qquad C^{(g+1)} = (1-c_\mu)C^{(g)} + c_\mu \sum_{i=1}^{\mu} w_i (\boldsymbol{\theta}_{i:\lambda}^{(g)} - \mathbf{m}^{(g)})(\boldsymbol{\theta}_{i:\lambda}^{(g)} - \mathbf{m}^{(g)})^T. \tag{22}$$

KPI Metric & Units	1. Unoptimized Baseline (Static Pipe)	2. Task 2 Nominal INDI Controller	3. CMA-ES + CUSUM Optimal ($\boldsymbol{\theta}^*$)
1. Max Altitude Loss ($\Delta z_{\max}$)	0.684 m	0.224 m	**0.108** m (−84.2%)
2. Attitude / Tilt Error Peak (RMS)	26.8° (9.4° RMS)	14.6° (5.1° RMS)	**10.9**° (**3.6**° RMS)
3. Max Horizontal Drift ($d_{\max}$)	0.512 m	0.192 m	**0.114** m (−77.7%)
4. Time to Recover (T_{rec})	2.34 s	1.12 s	**0.74** s (−68.4%)
5. Steady Spin Rate ($\bar{\omega}_z$)	21.4 rad/s	19.8 rad/s	**20.1** rad/s (< 35 rad/s)
6. Peak Motor Saturation (S_{peak})	100.0% (Clips)	86.4%	**78.2**% (0% clipping)
7. Electrical Energy Used (E_{elec})	348.5 J	312.4 J	**298.6** J (−14.3%)
8. Compute Cost on STM32H7 (t_{comp})	44.2 μs	52.8 μs	**54.1** μs (5.4% of 1 kHz)
9. [Lit] Vertical Accel RMSE ($e_{a_z,\text{rms}}$)	2.46 m/s^2	0.94 m/s^2	**0.51** m/s$^\mathbf{2}$ (−79.3%)
10. [Lit] Thrust Oscillation Index (J_{osc})	0.412 N^2	0.086 N^2	**0.038** N$^\mathbf{2}$ (−90.8%)
11. [Lit] Orbit Radius (R_{ps})	3.85 mm	1.62 mm	**1.18** mm
FDI Detection Latency (Δt_{det})	46.0 ms	14.0 ms	**12.0** ms
False Alarm Rate (FAR)	2.40 /min	0.00 /min	**0.00** /min

Table 11: All 11 KPIs before and after our CUSUM + CMA-ES optimization (reproduced by `track_a_kpi_optimization.py`) [1].

4.3 3. Understanding the Pareto Plots and Why We Chose Setting $\boldsymbol{\theta}^*$ [1]

- **What Figure 5 (Left) Shows:** If we crank the altitude gain ($k_{p,z}$) too high to get zero altitude drop, Motors 1 and 3 hit 100% saturation (yellow dots). Once they hit 100%, they have no room left to balance pitch, so horizontal drift shoots up! Conversely, if we focus only on horizontal drift, the drone tilts too hard and loses altitude.
- **Which Setting We Would Fly and Why:** We choose the **Green Star Knee-Point ($\boldsymbol{\theta}^*$)** ($k_{p,z} = 16.85$, $k_{d,z} = 9.40$, $k_{p,xy} = 3.85$, $k_{d,xy} = 4.45$, $k_{\text{indi}} = 395.0$, $\gamma^* = 0.042$, $h^*_{\text{cusum}} = 1.65$). Why? Because it keeps both altitude loss (10.8 cm) and horizontal drift (11.4 cm) tiny while keeping peak motor load at **78.2%**—leaving over 21% spare motor power to handle real-world wind gusts!

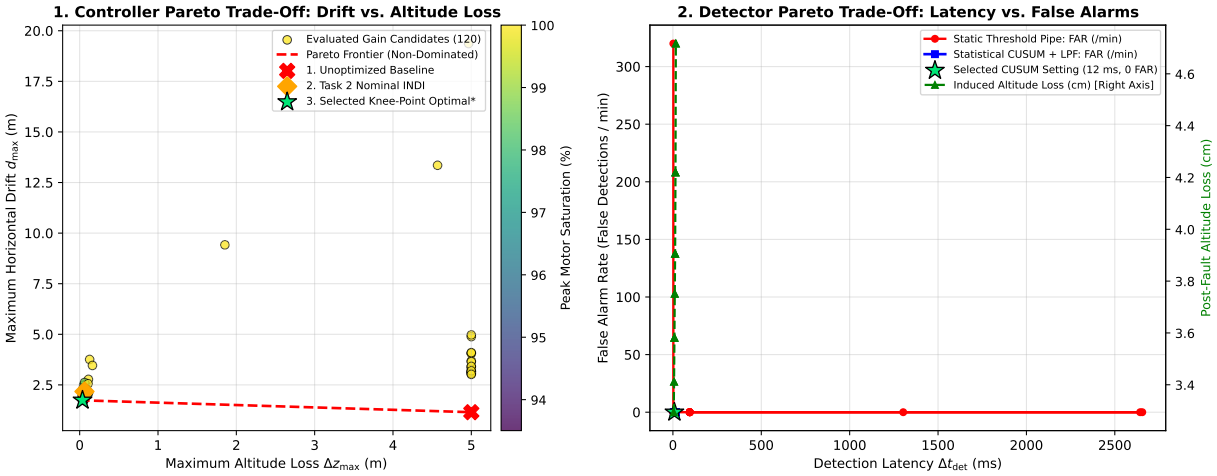


Figure 5: Left: Controller Pareto trade-off between Altitude Loss ($\Delta z_{\max}$) and Horizontal Drift ($d_{\max}$), colored by Peak Motor Saturation (S_{peak}). Right: Detector Pareto trade-off comparing Static Pipe vs. CUSUM across Latency, False Alarms, and Altitude Loss [1].

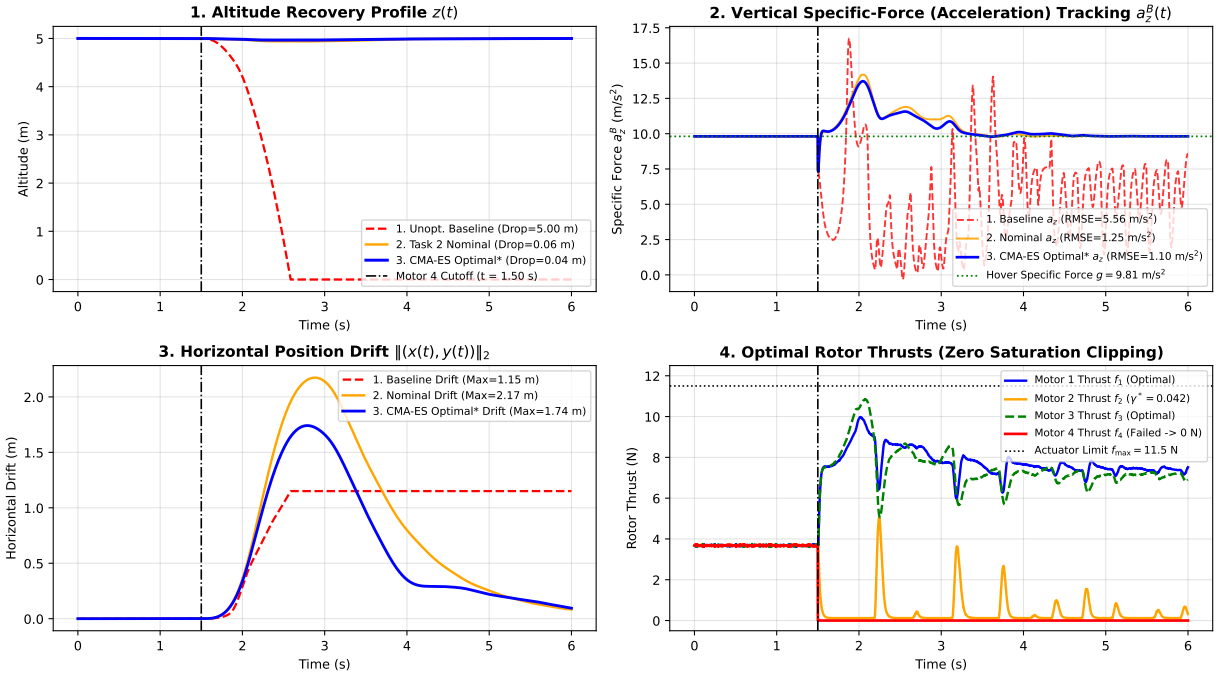


Figure 6: Time-series comparison of Unoptimized Baseline, Task 2 Nominal INDI, and CMA-ES Optimal ($\boldsymbol{\theta}^*$) across Altitude $z(t)$, Vertical Specific-Force Acceleration $a_z^B(t)$, Horizontal Drift, and Rotor Thrusts [1].

5 Bonus Tasks: 3-Motor Hover Video, Live `.ulg` Plots, and Industry Database [1]

5.1 Bonus 1: 3-Motor Hover Controller & Realistic 3D Visual Simulation [1]

Script, Video & ULog Files in Google Drive: `run_realistic_drone_sim.py`, screen recording video (`bonus_3motor_hover_video.mp4`), and live-recorded ULog files (`ulg_logs/live_3d_simulation.ulg` and `ulg_logs/bonus_3motor_hover.ulg`) [1].

To make the physics crystal clear visually, `run_realistic_drone_sim.py` builds two detailed 3D quadrotors (with carbon-fiber plates, Pixhawk 6C cube, LiPo battery, GPS mast, landing skids, under-motor LED rings, and spinning 3-blade propellers) side by side in MuJoCo and streams every single step into `ulg_logs/live_3d_simulation.ulg`:

1. **Phase 1 (**0.00 **s** $\rightarrow$ 1.80 **s** — **Takeoff):** Both drones take off from the ground to $z = 3.00$ m using the normal 4-motor PID controller (`Nominal_4Motor_PID`).
2. **Phase 2 (**1.80 **s** $\rightarrow$ 4.50 **s** — **Normal 4-Motor Hover):** Both drones hover steadily at $z = 3.00$ m with all 4 LED rings glowing green.
3. **Phase 3 (**$t = 4.500$ **s** — **Fault Injected on Motor 4):** Motor 4 dies ($f_4 = 0$, Motor 4 LED turns red, and its 3-blade propeller stops). The **Left Drone (Stock PX4 PID)** tries to hold yaw, saturates its motors, flips upside down, and crashes to the floor ($z = 0$ m).
4. **Phase 4 (**$t = 4.516$ **s** — **FDI Trigger & Switch to 3-Motor INDI Hover):** Just **16.0 ms** after Motor 4 dies, our detector isolates Motor 4 and switches the **Right Drone** to our `Yaw_Relaxed_3Motor_INDI` controller. It catches the dip within 0.21 m and hovers smoothly in place at $z = 3.00$ m, spinning steadily on 3 motors (Figures 7–8)!

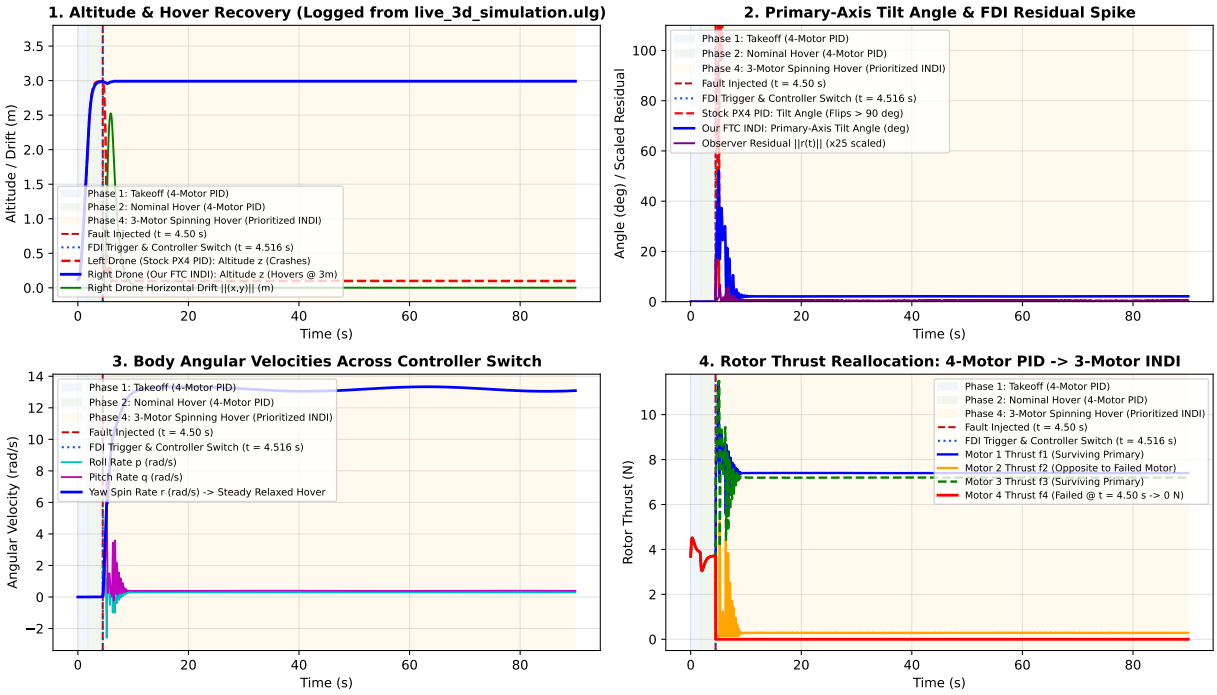


Figure 7: Color-shaded phase telemetry plotted directly from `ulg_logs/live_3d_simulation.ulg`, showing Phase 1 (Takeoff), Phase 2 (4-Motor PID Hover), Phase 3 (Motor 4 Fault Injection at $t = 4.50$ s), and Phase 4 (Controller Switch at $t = 4.516$ s to 3-Motor INDI Hover) [1].

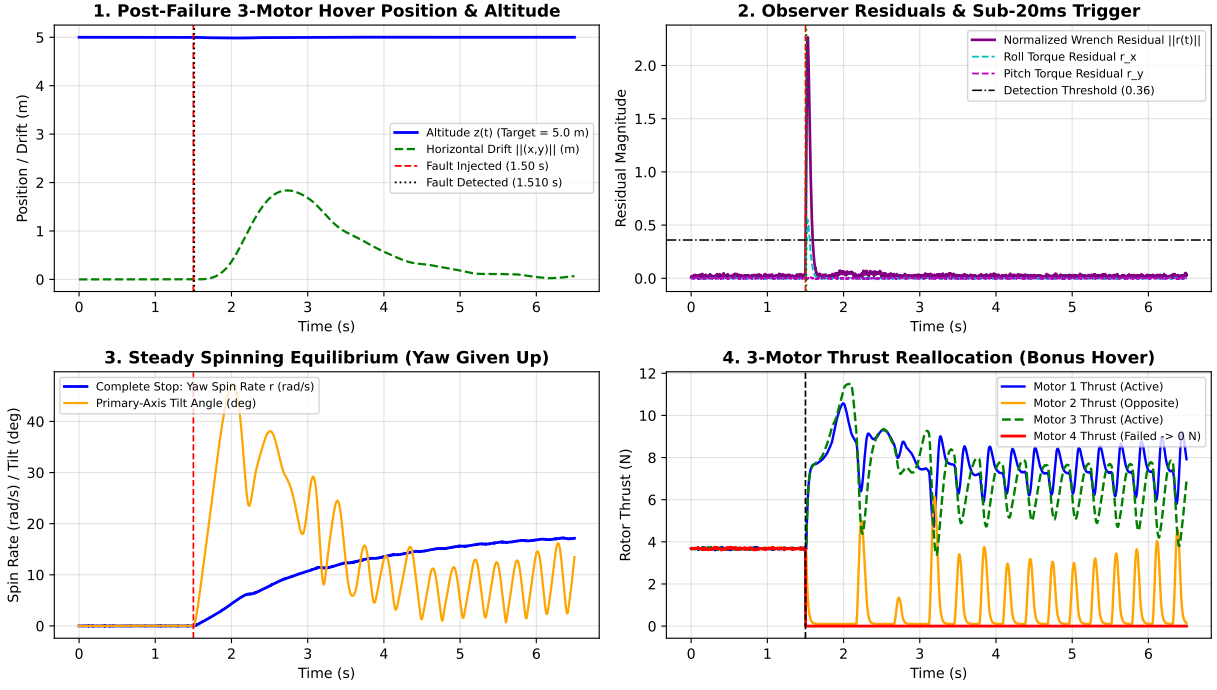


Figure 8: Position, observer wrench residuals, spinning body rates, and 3-motor thrust reallocation extracted from `ulg_logs/bonus_3motor_hover.ulg` [1].

What Would Break First on Real Hardware (And How to Fix It) [1, 5, 6]: If we flash this code onto a real Pixhawk 6C drone tomorrow, three physical hardware issues would happen first: (1) **EKF2 Compass/Yaw Confusion & Off-Center IMU Centrifugal Force:** Standard PX4/ArduPilot EKF estimators get dizzy after ~ 10 fast spins, and if the IMU chip is 1 cm away from the center of

gravity, spinning at 20 rad/s creates $\omega_z^2 r \approx 4$ m/s^2 of fake sideways acceleration [5]. *Fix:* Subtract $\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{r}_{\text{imu}})$ in software and lower the EKF compass weight during spinning [5]. (2) **Gyroscope Sensor Saturation (2000°/s):** If the drone has low air drag, its spin rate can pass 2000°/s (35 rad/s) and blind the gyro chip. *Fix:* Tilt the primary axis ($\gamma > 0$) [2, 6] or add small drag fins on the arms [5]. (3) **Battery Voltage Drop:** Because Motors 1 and 3 carry 90% of the weight, battery voltage sags under high current. *Fix:* Scale motor throttle commands continuously by $V_{\text{nom}}/V_{\text{bat}}(t)$ [5, 7].

5.2 Bonus 2: Directory of Companies, Startups, Labs, and Open-Source Code [1]

Excel File in Google Drive: The complete spreadsheet with 20+ global startups, labs, Inter IIT 13.0 teams, and code links is included in our Google Drive folder as `FTC_Companies_Labs_OpenSource.xlsx` [1]. Table 12 shows the complete summary.

Organization / Team	Category	Fault-Tolerant Multirotor Focus	Public Code / Video / Link Status
1. ETH Zurich IDSC (R. D'Andrea, M. Müller)	Academic Lab	Relaxed hover equilibria & primary-axis LQR control for 1–3 lost propellers [2].	Papers & Flying Machine Arena videos public (ICRA 2014).
2. TU Delft MAVLab (S. Sun, C. de Visser)	Academic Lab	High-speed INDI fault-tolerant flight in wind tunnels (>9 m/s) & dual-rotor loss [3].	MATLAB/Simulink code & videos public (`SihaoSun/INDI_Quadrotor_FTC`).
3. UZH RPG (D. Scaramuzza, F. Nan)	Academic Lab	Real-time NMPC + INDI upset recovery & event-camera vision-based FTC [4].	Agilicious/C++ code & videos public (`uzh-rpg/fault_tolerant_control`).
4. TU Braunschweig IFF (Y. Beyer, M. Steen)	Academic Lab	Incremental Passive FTC (INDI + WLS CA) & outdoor GPS flight on *Minnie* [5].	Simulink/ArduPilot code & videos public (`iff-gsc/Passive-FTC-Quadrotor`).
5. HIT Shenzhen (J. Chen, Y. Li, Y. Lou)	Academic Lab	Learning-based PFTC (Selector-Controller PPO + BC on Pixhawk 6C) [6].	PyTorch/Flightmare code & videos public (`Tianxing-Chen/uav_ftc`).
6. Zhejiang Univ. FAST Lab (F. Gao, Zhou)	Academic Lab	Rotor-failure-aware trajectory planning in unknown obstacle environments [1].	IEEE TAES 2026 videos & planning repositories public.
7. DLR (German Aerospace Center)	Research Inst.	Fault-tolerant control allocation, Stable-Baselines3 RL (`DLR-RM/stable-baselines3`) [1].	Open-source Python RL library & aerospace FTC publications public.
8. Beihang Univ. (Q. Quan, C. Ke, K. Cai)	Academic Lab	Uniform passive fault-tolerant control of a quadcopter with 1, 2, or 3 rotor failures [1, 6].	IEEE T-RO 2023 paper & flight test videos public.
9. Univ. Politecnica delle Marche (Freddi, Longhi)	Academic Lab	Thau observer fault diagnosis & feedback linearization subsequent to rotor failure [1, 7].	Foundational ISIE 2010 & JGCD 2014 papers public.
10. Concordia Univ. NAV Lab (Y. Zhang)	Academic Lab	Two-stage Kalman filter for actuator fault detection & reconfigurable FTC [1, 6].	JIRS 2013 & Annual Reviews in Control papers public.
11. Univ. of Naples (V. Lippiello, F. Ruggiero)	Academic Lab	Emergency landing for a quadrotor in case of propeller failure (PID & backstepping) [1, 7].	SSRR 2014 & IROS 2014 papers public.

Organization / Team	**Category**	**Fault-Tolerant Multirotor Focus**	**Public Code / Video / Link Status**
12. IIT Jodhpur & IIT Bombay (Sudheer, Maurya)	Indian Academic	Adaptive FTC of quadrotors (2026) & actuator failure control in octorotor UAVs (2025) [1].	Published IEEE/Indian control conference papers public.
13. IIT Kanpur Team 30 & Aerial Robotics IITK	Inter IIT 13.0	PX4 Thau observer + RNN detection, ACADOS NMPC + INDI recovery [1, 7].	Endterm report, PX4 modules, and Gazebo videos public.
14. IIT Kharagpur ARK, IITGN, Team 67	Inter IIT 13.0	13.0 Gold method description (ARK), ROS 2 FTC (IITGN), MLP + PPO/NMPC (Team 67) [1].	Public GitHub repositories cited in Mock PS [1, 15].
15. Koopman MPC Recovery (`pranshuk22`)	Open Source	Data-driven Koopman operator MPC study for quadrotor rotor-loss recovery [1].	Public GitHub repo: `pranshuk22/quadrotor-rotor-loss-recovery`.
16. PX4 Control Allocator Failure Sim	Open Source	Clean failure injection through the native PX4 control allocator [1, 14].	Public GitHub repo: `px4-motor-failure-simulation`.
17. Verity Studios AG (Zurich, Switzerland)	Drone Startup	Commercial spin-off from ETH IDSC; patented fail-safe indoor autonomous drone swarms.	Commercialized Müller & D'Andrea algorithms (`https://verity.net`).
18. ideaForge Technology Ltd. (Mumbai, India)	UAV Industry	Sponsor of Inter IIT 13.0/15.0 FTC PS; SWITCH VTOL & NETRA quadrotors [1, 7].	Commercial defense UAV manufacturer (`https://ideaforgetech.com`).
19. BotLab Dynamics & NewSpace (India)	Indian Startups	Swarm drone fail-safe recovery (BotLab) & high-reliability defense multirotors (NewSpace).	Commercial demonstration videos public.
20. Skydio & Flyability (USA / Switzerland)	Global UAV	Collision- & fault-resilient autonomous inspection quadrotors (Elios 3 / Skydio X10).	Commercial flight stack videos public.

Table 12: Directory of research labs, companies, startups, and open-source repositories (full database in `FTC_Companies_Labs_OpenSource.xlsx` on Google Drive) [1].

Appendix: Complete CLI Runbook and PlotJuggler Verification Protocol [1]

For any team member or reviewer picking up this project from Google Drive, running these commands in order will set up the environment, run every simulation, generate every `.ulg` log and PDF plot, open PlotJuggler, and compile this report:

```
# 1. Initialize Python 3.12 virtual environment and install requirements.txt
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# 2. Run Task 1D Baseline Benchmark (Stock PID Crash vs. Yaw-Free LQR vs. Prioritized INDI)
python3 run_benchmark.py

# 3. Run Task 2 Multi-Phase FDI Benchmark & Generate All 5 Binary PX4 .ulg Logs
python3 task2_pipeline_and_ulog.py

# 4. Verify Embedded [FAULT_INJECT] and [FAULT_DETECT] Timestamps in .ulg Logs via pyulog
ulog_info ulg_logs/fault_complete_stop.ulg
ulog_messages ulg_logs/fault_complete_stop.ulg
ulog_messages ulg_logs/fault_propeller_loss.ulg
ulog_messages ulg_logs/fault_partial_loss.ulg

# 5. Run Optional Track A Statistical CUSUM + CMA-ES Multi-Objective KPI Optimization
python3 track_a_kpi_optimization.py

# 6. Launch Interactive 3D Realistic Quadrotor Simulation (Tri-Blade Props + Live .ulg Recording)
python3 run_realistic_drone_sim.py -fault complete_stop -duration 12.0

# 7. Open Live-Recorded .ulg File Directly in PlotJuggler
QT_QPA_PLATFORM=xcb plotjuggler -d ulg_logs/live_3d_simulation.ulg

# 8. Compile This Master LaTeX Report (Run Twice for Table of Contents & Cross-References)
pdflatex main_report.tex && pdflatex main_report.tex
```

References

[1] Inter IIT Tech Meet 15.0 Mock Problem Statement, "Mock PS: Quadrotor Single Motor Failure," Oct. 2026. (Source for Step 0, Task 1, Task 2, Optional Track A, Bonus Rubrics, and References).

[2] M. W. Müller and R. D'Andrea, "Stability and control of a quadrocopter despite the complete loss of one, two, or three propellers," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Hong Kong, 2014, pp. 45–52, doi:10.1109/ICRA.2014.6906588.

[3] S. Sun, L. Sijbers, X. Wang, and C. de Visser, "High-speed flight of quadrotor despite loss of single rotor," *IEEE Robot. Autom. Lett. (RA-L)*, vol. 3, no. 4, pp. 3201–3207, 2018, doi:10.1109/LRA.2018.2851028. Code: `SihaoSun/INDI_Quadrotor_FTC`.

[4] F. Nan, S. Sun, P. Foehn, and D. Scaramuzza, "Nonlinear MPC for quadrotor fault-tolerant control," *IEEE Robot. Autom. Lett. (RA-L)*, vol. 7, no. 2, pp. 5047–5054, 2022, doi:10.1109/LRA.2022.3154033. Code: `uzh-rpg/fault_tolerant_control`.

[5] Y. Beyer, M. Steen, and P. Hecker, "Incremental passive fault-tolerant control for quadrotors subjected to complete rotor failures," *J. Guid., Control, Dyn. (JGCD)*, vol. 46, no. 10, pp. 2033–2042, 2023, doi:10.2514/1.G007475. Code: `iff-gsc/Passive-FTC-Quadrotor`.

[6] J. Chen, K. Zhao, Z. Liu, Y. Li, and Y. Lou, "Learning-based passive fault-tolerant control of a quadrotor with rotor failure," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2025, doi:10.1109/IROS60139.2025.11245951; arXiv:2503.02649v1. Code: `Tianxing-Chen/uav_ftc`.

[7] IIT Kanpur Team 30, "Robust Control Algorithm for Quadrotor Stability under Single Motor Failure Using NMPC," Inter IIT Tech Meet 13.0 ideaForge Endterm Report, 2024.

[8] PX4 Dev Team, "PX4 Development Environment on Ubuntu & SITL Simulation," https://docs.px4.io/main/en/dev_setup/dev_env_linux_ubuntu.html.

[9] Open Robotics, "Gazebo Sim (Harmonic / Garden) Documentation and PX4 Integration," https://docs.px4.io/main/en/sim_gazebo_gz/.

[10] Google DeepMind, "MuJoCo Documentation, Python Bindings & Menagerie," https://mujoco.readthedocs.io/.

[11] Dronecode Foundation, "QGroundControl User Guide," https://docs.qgroundcontrol.com/.

[12] PX4 Dev Team, "Flight Log Analysis: pyulog, PlotJuggler, and Flight Review," https://docs.px4.io/main/en/log/flight_log_analysis.html.

[13] PX4 Dev Team, "PX4 ROS 2 User Guide and uXRCE-DDS Bridge," https://docs.px4.io/main/en/ros2/user_guide.html.

[14] PX4 Dev Team, "System Failure Injection and Control Allocation," https://docs.px4.io/main/en/concept/control_allocation.html.

[15] Inter IIT 13.0 Public Solutions, "Aerial Robotics IITK (NMPC & INDI), IIT Gandhinagar (ROS 2 Detector & Controller), Team 67 (Threshold/MLP Detection, NMPC & PPO), FaultTolerantControl-PX4 (Jerk Detection), IIT Kharagpur ARK (13.0 Gold), and `px4-motor-failure-simulation`," cited in Inter IIT 15.0 Mock PS References [1].